

System Administration & Security Report

Coursework Submission (Part 1 Logbook + Part 2 Security Hardening Report)

Table of Content

COMP1475 – Part 1 Logbook

1. Lab 3 – Web Service Management (Apache HTTP Server)..... 3
2. Lab 4 – Database Service Management (MariaDB)..... 13
3. Lab 6 – Firewall Configuration..... 26

COMP1475 – Part 2

Apache HTTP default service configuration (Primary Focus)

Linux host firewall default configuration (Secondary Focus)

1. INTRODUCTION.....	36
2. LITERATURE REVIEW.....	39
3. VULNERABILITY ANALYSIS.....	42
3.1 Apache Vulnerability Findings.....	42
3.2 Supporting Firewall Vulnerability Findings.....	48
4. HARDENING PROCEDURE.....	51
4.1 H1 — Minimize Server Identification.....	51
4.2 H2 — Apply Security Headers.....	53
4.3 H3 — Disable Directory Listing & Remove autoindex.....	54
4.4 H4 — Enable TLS, Disable Legacy Protocols, Apply HSTS & Redirect HTTP→HTTPS.....	57
4.5 H5 —Disable TRACE and Restrict HTTP Methods.....	60
4.6 H6 — Remove Unnecessary Modules: autoindex & status.....	61
4.7 H7 — Host Firewall Hardening.....	62
5. RISK ASSESMENT.....	65
5.1 H1 — Apache HTTP Server Risks.....	65
5.2 Firewall Exposure Risks (Secondary Layer).....	66
5.3 Risk Assessment Methodology.....	69
6. RESULT ANALYSIS.....	75
6.1 Improvements.....	75
6.2 Result Interpretation Table.....	75
6.3 Interpretation of Improvements.....	77
6.4 Overall Security Posture After Hardening.....	77
7. CONCLUSIONS & REFLECTIONS.....	78
8. REFERENCES.....	80

COMP1475 – Part 1 Logbook

Lab 3 – Web Service Management (Apache HTTP Server)

Student: Apurba Sarker

Module: COMP1475 – System Administration & Security

Academic Year: 2025–2026

1. Introduction

This lab presented the installation, management, and monitoring of Apache HTTP Server (httpd) on a Linux system based on CentOS 8. The aim was to learn the behavior of Linux services in systemd, the structure of Apache processes and the configuration files which specify the behavior of key services including ports, document root, logging, and separation of privileges. By undertaking this lab, I enhanced my knowledge about service administration, verification, and simple considerations about web server security.

2. Lab Tasks and Solutions

2.1 Root Access

To start, I elevated privileges and moved to the filesystem root directory:

```
apurba@localhost:~$ sudo -i
[sudo] password for apurba:
root@localhost:~# cd /
pwd
/
root@localhost:/# whoami
root
root@localhost:/#
```

Figure 1: root access

This ensured that I gained root access, which is needed to install and configure the system services.

2.2 Searching for Apache Packages

I confirmed that the Apache packages was available in the following system repositories:

```
yum search httpd | less
```

```
===== Name Exactly Matched: httpd =====
httpd.x86_64 : Apache HTTP Server
===== Name & Summary Matched: httpd =====
centos-logos-httpd.noarch : CentOS-related icons and pictures used by httpd
httpd-core.x86_64 : httpd minimal core
keycloak-httpd-client-install.noarch : Tools to configure Apache HTTPD as Keycloak client
python3-keycloak-httpd-client-install.noarch : Tools to configure Apache HTTPD as Keycloak client
===== Name Matched: httpd =====
httpd-devel.x86_64 : Development interfaces for the Apache HTTP Server
httpd-filesystem.noarch : The basic directory layout for the Apache HTTP Server
httpd-manual.noarch : Documentation for the Apache HTTP Server
httpd-tools.x86_64 : Tools for use with the Apache HTTP Server
libmicrohttpd.x86_64 : Lightweight library for embedding a webserver in applications
===== Summary Matched: httpd =====
mod_dav_svn.x86_64 : Apache httpd module for Subversion server
mod_proxy_cluster.x86_64 : JBoss mod_proxy_cluster for Apache httpd
~
```

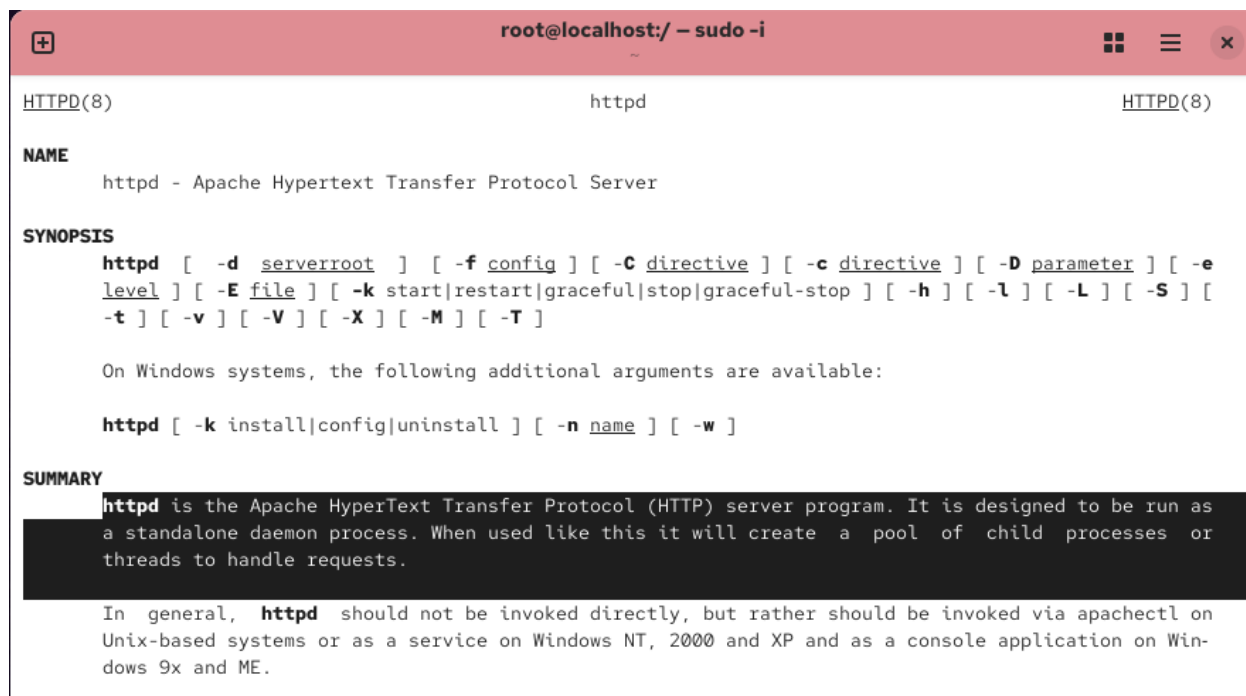
Figure 2: Available Apache packages

This output displayed all httpd-related packages, including the primary httpd.x86_64 package. less permitted scrolling through the list safely without altering content.

2.3 Apache manual viewing

I confirmed that I accessed Apache's manual entry to check its purpose and usage.

```
man httpd
```



```
root@localhost:/ - sudo -i

HTTPD(8)                                httpd                                HTTPD(8)

NAME
    httpd - Apache Hypertext Transfer Protocol Server

SYNOPSIS
    httpd [ -d serverroot ] [ -f config ] [ -C directive ] [ -c directive ] [ -D parameter ] [ -e
    level ] [ -E file ] [ -k start|restart|graceful|stop|graceful-stop ] [ -h ] [ -l ] [ -L ] [ -S ] [
    -t ] [ -v ] [ -V ] [ -X ] [ -M ] [ -T ]

    On Windows systems, the following additional arguments are available:

    httpd [ -k install|config|uninstall ] [ -n name ] [ -w ]

SUMMARY
    httpd is the Apache HyperText Transfer Protocol (HTTP) server program. It is designed to be run as
    a standalone daemon process. When used like this it will create a pool of child processes or
    threads to handle requests.

    In general, httpd should not be invoked directly, but rather should be invoked via apachectl on
    Unix-based systems or as a service on Windows NT, 2000 and XP and as a console application on Win-
    dows 9x and ME.
```

Figure 3: Httpd manual

The demonstration focuses on that the server daemon should not be started directly. Rather, administrators should depend on systemctl or service scripts to confirm correct configuration loading, logging, and process listing.

2.4 Apache status check

The command below was used to monitor active httpd process:

```
ps -ef | grep httpd
```

```

root@localhost:/# ps -ef | grep httpd
root      1306      1  0 14:17 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    4326    1306  0 14:24 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    4328    1306  0 14:24 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    4329    1306  0 14:24 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    4350    1306  0 14:24 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
root      4797    4683  0 14:30 pts/1    00:00:00 grep --color=auto httpd
root@localhost:/#

```

Figure 4: Active Apache processes

The output highlights

- One root inherited parent process
- Multiple worker processes running as the Apache user

This demonstrated that Apache's multi-process model, include the master process handles privileged operations and worker processes handle client requests under restricted privileges.

2.5 Apache Services starting and verifications after

```
systemctl start httpd
```

```
systemctl status httpd
```

```

root@localhost:/# systemctl start httpd
root@localhost:/# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-03-24 14:18:04 GMT; 13min ago
 Invocation: 1a1249914c1a45d393be29e5d4413834
    Docs: man:httpd.service(8)
  Process: 4322 ExecReload=/usr/sbin/httpd $OPTIONS -k graceful (code=exited, status=0/SUCCESS)
 Main PID: 1306 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 10316)
  Memory: 18.7M (peak: 22.5M, swap: 0B, swap peak: 4K)
     CPU: 2.361s

```

Figure 5: Apache status

The output ensures:

- The service was active (running)
- Loaded from /usr/lib/systemd/system/httpd.service
- The main PID was functioning normally

2.6 Service Startup Behavior check

I used following commands to verify Apache would start automatically at boot.

```
systemctl is-enabled httpd
```

```
systemctl get-default
```

```
root@localhost:/# systemctl is-enabled httpd
enabled
root@localhost:/# systemctl get-default
graphical.target
root@localhost:/#
```

Figure 6: startup check

The output confirms:

- Apache was enabled as it will start the system boots.
- The default system target was **graphical.target**, that confirms multi-user mode and services such as web servers.

2.7 Checking the Systemd Unit File

Apache's systemd service definition was checked:

```
less /usr/lib/systemd/system/httpd.service
```

The key configuration line:

```
ExecStart=/usr/sbin/httpd -DFOREGROUND
```

```
apurba@localhost:~ -- less /usr/lib/systemd/system/httpd....

# See httpd.service(8) for more information on using the httpd service.

# Modifying this file in-place is not recommended, because changes
# will be overwritten during package upgrades. To customize the
# behaviour, run "systemctl edit httpd" to create an override unit.

# For example, to pass additional options (such as -D definitions) to
# the httpd binary at startup, create an override unit (as is done by
# systemctl edit) and enter the following:

#         [Service]
#         Environment=OPTIONS=-DMY_DEFINE

[Unit]
Description=The Apache HTTP Server
Wants=httpd-init.service
After=network.target remote-fs.target nss-lookup.target httpd-init.service
Documentation=man:httpd.service(8)

[Service]
Type=notify
Environment=LANG=C

ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND
ExecReload=/usr/sbin/httpd $OPTIONS -k graceful
# Send SIGWINCH for graceful stop
KillSignal=SIGWINCH
KillMode=mixed
:
```

Figure 7: System unit file

This confirmed how systemd launches the web server in the foreground, permitting systemd to monitor and control it.

2.8 Checking Apache's Main Configuration File

I used the following commands:

```
find / -name httpd.conf
less /etc/httpd/conf/httpd.conf
```


I checked the main Apache configuration file and identified essential directives:

- **ServerRoot**

```
..
# ServerRoot: The top of the directory tree under which the server's
# configuration, error, and log files are kept.
#
# Do not add a slash at the end of the directory path. If you point
# ServerRoot at a non-local disk, be sure to specify a local disk on the
# Mutex directive, if file-based mutexes are used. If you wish to share the
# same ServerRoot for multiple httpd daemons, you will need to change at
# least PidFile.
#
ServerRoot "/etc/httpd"

# ServerAdmin: Your address, where problems with the server should be
# e-mailed. This address appears on some server-generated pages, such
# as error documents. e.g. admin@your-domain.com
#
ServerAdmin root@localhost

#
# ServerName gives the name and port that the server uses to identify itself.
# This can often be determined automatically, but we recommend you specify
# it explicitly to prevent problems during startup.
#
# If your host doesn't have a registered DNS name, enter its IP address here.
#
#ServerName www.example.com:80
```

Figure 8: server root

- **User and group**

```
..
User apache
Group apache

# 'Main' server configuration
#
# The directives in this section set up the values used by the 'main'
# server, which responds to any requests that aren't handled by a
# <VirtualHost> definition. These values also provide defaults for
# any <VirtualHost> containers you may define later in the file.
#
# All of these directives may appear inside <VirtualHost> containers,
# in which case these default settings will be overridden for the
# virtual host being defined.
#
```

Figure 9: User and group

- **Listening Port**

```
# Change this to Listen on a specific IP address, but note that if
# httpd.service is enabled to run at boot time, the address may not be
# available when the service starts. See the httpd.service(8) man
# page for more information.
#
#Listen 12.34.56.78:80
Listen 80
```

Figure 10: Listening port

- **Logging Locations**

```
logs/error_log
logs/access_log
```

3. Answers to Apache Configuration questions

The required answers highlighting inspection of httpd.conf and system outputs are stated below:

1. Include directives present?

Yes. Apache includes in the directives of its httpd.conf include directives which enable the server to automatically load other configuration files: IncludeOptional conf.modules.d/.conf IncludeOptional conf.d/.conf. This makes the major configuration modular and allows the administrator to handle settings in separate, organised files like /etc/httpd/conf.d/welcome.conf.

2. ServerRoot value and directory?

The ServerRoot will be /etc/httpd that serves as the root of Apache. Every relative path, such as configuration, logs and module files, is resolved relative to this point, so that the core files of Apache are still structured under one directory hierarchy.

3. Less-privileged user and group for worker processes?

Apache implements its worker processes as apache user and group. Though master process initiates as root to bind to port 80, the worker processes switch privileges and execute as apache, being less risky in that the processes can do less in case of attack.

4. Why hide .htaccess and .htpasswd from clients?

Access rules and hashed passwords are sensitive information that is stored in htaccess and .htpasswd. This would be disclosing vital information on security and thus such files should remain undisclosed in a way that information leakage and unauthorized access by unauthorized persons can be avoided.

5. Where is the error log written?

The error log is configured to be logs/errorlog which has a value of /var/log/httpd/errorlog. This file captures warnings and server errors and is necessary in troubleshooting configuration problems and run time problems.

6. Where to place additional Apache configuration instructions?

Further configuration would be contained in /etc/httpd/conf.d/ as individual.conf files. Apache is configured to load all the files in this directory automatically, thus clean, modular configuration is made possible without altering the main httpd.conf.

7. Which port(s) does Apache listen on?

The apache server is configured to listen on a port of 80 which is shown by the Listen 80 command. This is the default port of HTTP traffic.

8. What is the DocumentRoot?

DocumentRoot is the default directory which is /var/www/html where Apache delivers the files of the websites to its users.

9. Where is the access log located?

Access log is stored in logs/accesslog which is found to be /var/log/httpd/accesslog. It logs every request sent through the server.

10. Difference between access and error logs?

Access log documents every client request with IP, method, URL and response code whereas error log documents server-side problems like errors in configuration, warnings and run time crashes.

11. After starting Apache, does it appear in the process list and do owners match the config?

Yes. The process list indicates that there is one parent process that is owned by the root and multiple worker processes that are running as the apache user which is just as per the Apache configuration.

4. Explanation

This lab highlights that it is crucial to learn the mechanisms of interaction of system services with internal Linux environment. The architecture of Apache reflects some of the major administrative and security concepts. Firstly, its support of systemd gives it formal lifecycle management of services, predictable startup behavior, reliable logging, automatic recovery when required, and controlled shutdown. Secondly, the absence of privilege separation in Apache allows separation of privileges between the master process (which operates as the root user) and the worker processes (which operate as an unprivileged user), which minimizes the attack surface to a large extent. Thirdly, it has a modular configuration system that ensures administrators to maintain big and intricate deployments in a clean and safe way. These aspects are critical in the deployment of secure and scalable web services.

5. Conclusion

This lab gave me a practical experience in managing a large-scale production grade web server. I also learned how crucial configuration files may be found and read, how to inspect the behavior of other services, and how the separation of privileges may be applied to improve security. I also learned the applicability of systemd in the existing Linux systems due to the availability of uniformity when managing services of different services. Log inspection, process ownership, and configuration instructions were useful in demonstrating how inappropriate configuration can lead to security vulnerability.

Background development of more complex tasks, such as configuring virtual hosts, deploying TLS, and hardening Apache, which play an important role in secure systems administration skills, could be developed in this lab.

Lab 4 – Database Service Management (MariaDB)

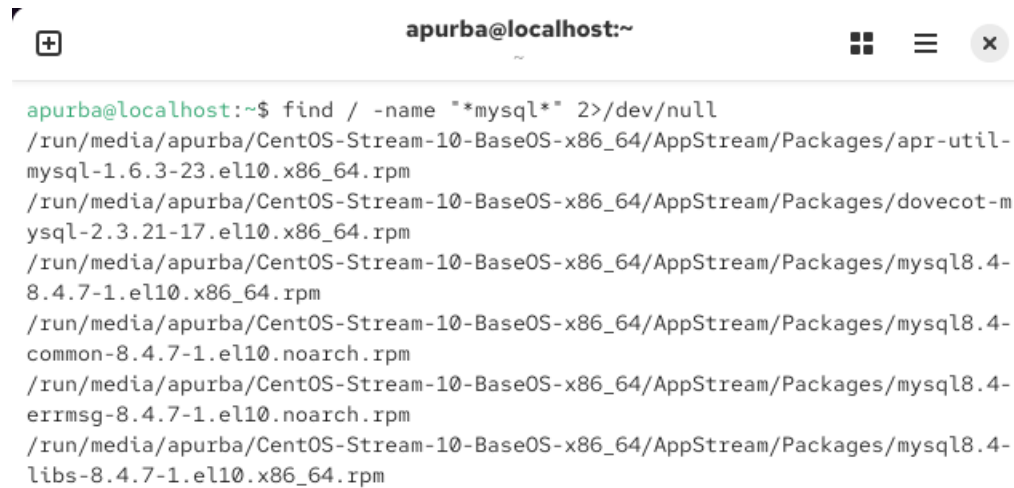
1. Introduction

The lab was founded on the MariaDB database server installation, configuration, and security on CentOS Linux platform. Most Linux operating systems have as their default database engine MariaDB that provides relational database services to LAMP based services. The objective of the lab was to check installation, keep the service running with systemd, check on the configuration settings, understand how MariaDB handles the data storage and apply the appropriate security practices such as user privilege controls and hardening. The performance of this lab provided an opportunity to have a practical experience of database administration and describe the fundamental concepts of security, such as the least privilege principle, hardening of authentication, and protection of filesystems.

2. Lab Tasks and Solutions

2.1 Checking for Existing MySQL Components

The lab started with identifying any existing MySQL-related files on the system:



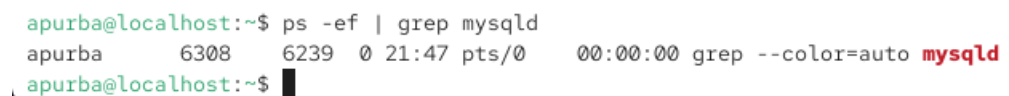
```
apurba@localhost:~$ find / -name "*mysql*" 2>/dev/null
/run/media/apurba/CentOS-Stream-10-BaseOS-x86_64/AppStream/Packages/apr-util-
mysql-1.6.3-23.el10.x86_64.rpm
/run/media/apurba/CentOS-Stream-10-BaseOS-x86_64/AppStream/Packages/dovecot-m
ysql-2.3.21-17.el10.x86_64.rpm
/run/media/apurba/CentOS-Stream-10-BaseOS-x86_64/AppStream/Packages/mysql8.4-
8.4.7-1.el10.x86_64.rpm
/run/media/apurba/CentOS-Stream-10-BaseOS-x86_64/AppStream/Packages/mysql8.4-
common-8.4.7-1.el10.noarch.rpm
/run/media/apurba/CentOS-Stream-10-BaseOS-x86_64/AppStream/Packages/mysql8.4-
errmsg-8.4.7-1.el10.noarch.rpm
/run/media/apurba/CentOS-Stream-10-BaseOS-x86_64/AppStream/Packages/mysql8.4-
libs-8.4.7-1.el10.x86_64.rpm
```

Figure 1: FindMysql

The output confirmed checks on various MySQL-related packages in mounted installation media, enabling MySQL components were available on the system but not installed as active services.

2.2 Database Server Status Check

To check if MariaDB was already active:



```
apurba@localhost:~$ ps -ef | grep mysqld
apurba      6308      6239  0 21:47 pts/0    00:00:00 grep --color=auto mysqld
apurba@localhost:~$
```

Figure 2: MariaDB status

The output highlights no mysqld process running except the grep command itself, highlighting that MariaDB/MySQL was not running.

2.3 Installing MariaDB Server

MariaDB and MariaDB server installation process include:

```

apurba@localhost:~$ sudo dnf install -y mariadb mariadb-server
[sudo] password for apurba:
Last metadata expiration check: 0:29:25 ago on Wed 25 Mar 2026 09:20:06 PM GMT.
Package mariadb-3:10.11.15-2.el10.x86_64 is already installed.
Package mariadb-server-3:10.11.15-2.el10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
apurba@localhost:~$

```

Figure 3: MariaDB Installation

The output highlights both packages as already present as installation was previously completed.

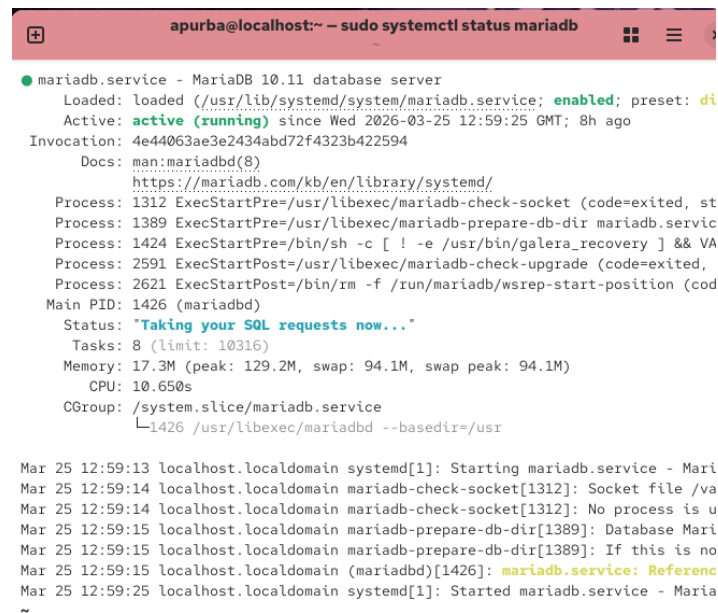
2.4 Database Service Starting

The MariaDB service started and verified by following command:

```

sudo systemctl start mariadb
sudo systemctl status mariadb

```



```

apurba@localhost:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-03-25 12:59:25 GMT; 8h ago
     Invocation: 4e44063ae3e2434abd72f4323b422594
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 1312 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
    Process: 1389 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
    Process: 1424 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VA (code=exited, status=0/SUCCESS)
    Process: 2591 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
    Process: 2621 ExecStartPost=/bin/rm -f /run/mariadb/wsrep-start-position (code=exited, status=0/SUCCESS)
   Main PID: 1426 (mariadbd)
      Status: "Taking your SQL requests now..."
         Tasks: 8 (limit: 10316)
        Memory: 17.3M (peak: 129.2M, swap: 94.1M, swap peak: 94.1M)
           CPU: 10.650s
      CGroup: /system.slice/mariadb.service
             └─1426 /usr/libexec/mariadbd --basedir=/usr

Mar 25 12:59:13 localhost.localdomain systemd[1]: Starting mariadb.service - MariaDB 10.11 database server: taking your SQL requests now...
Mar 25 12:59:14 localhost.localdomain mariadb-check-socket[1312]: Socket file /var/lib/mysql/mysql.sock already exists
Mar 25 12:59:14 localhost.localdomain mariadb-check-socket[1312]: No process is using the socket
Mar 25 12:59:15 localhost.localdomain mariadb-prepare-db-dir[1389]: Database Mari
Mar 25 12:59:15 localhost.localdomain mariadb-prepare-db-dir[1389]: If this is no
Mar 25 12:59:15 localhost.localdomain (mariadbd)[1426]: mariadb.service: Referenc
Mar 25 12:59:25 localhost.localdomain systemd[1]: Started mariadb.service - MariaDB 10.11 database server: taking your SQL requests now...

```

Figure 4: MariaDB active status

The service output shows **Active: active (running)**, with mariadb processing normally, confirming successful startup.

2.5 MariaDB System User Verification

MariaDB generates a dedicated system account to ensure security. This verification include:

```
apurba@localhost:~$ cat /etc/passwd | grep mysql
mysql:x:27:27:MySQL Server:/var/lib/mysql:/sbin/nologin
apurba@localhost:~$
```

Figure 5: Verification

The output presents the MySQL system user with a non-login shell, consistent with service account design security.

2.6 Securing the Database Installation

The `mysql_secure_installation` script was run in order to harden the database:

```
apurba@localhost:~$ sudo mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
      SERVERS IN PRODUCTION USE!  PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] n
... skipping.
```

Figure 6: Securing step 1


```
You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] n
... skipping.

You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n] y
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] y
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
```

Figure 6: Securing step 2

Key hardening steps performed:

- Updated root password
- Removed anonymous users
- Disabled remote root login
- Removed test database
- Reloaded tables privilege

These modifications significantly mitigate attack surface by enabling strong authentication and removing insecure defaults.

```
apurba@localhost:~  
Normally, root should only be allowed to connect from 'localhost'. This  
ensures that someone cannot guess at the root password from the network.  
  
Disallow root login remotely? [Y/n] y  
... Success!  
  
By default, MariaDB comes with a database named 'test' that anyone can  
access. This is also intended only for testing, and should be removed  
before moving into a production environment.  
  
Remove test database and access to it? [Y/n] y  
- Dropping test database...  
... Success!  
- Removing privileges on test database...  
... Success!  
  
Reloading the privilege tables will ensure that all changes made so far  
will take effect immediately.  
  
Reload privilege tables now? [Y/n] y  
... Success!  
  
Cleaning up...  
  
All done! If you've completed all of the above steps, your MariaDB  
installation should now be secure.  
  
Thanks for using MariaDB!  
apurba@localhost:~$
```

Figure 7: Securing step 3

2.8 Database Data Directory Locating

MariaDB locates all databases under:

```
/var/lib/mysql
```

The directory was checked with the followings:

```
apurba@localhost:~$ ls -l /var/lib/mysql
total 123324
-rw-rw----. 1 mysql mysql 417792 Mar 25 21:54 aria_log.000000001
-rw-rw----. 1 mysql mysql 52 Mar 20 00:31 aria_log_control
-rw-rw----. 1 mysql mysql 9 Mar 20 00:51 ddl_recovery-backup.log
-rw-rw----. 1 mysql mysql 9 Mar 20 00:51 ddl_recovery.log
-rw-rw----. 1 mysql mysql 906 Mar 19 13:31 ib_buffer_pool
-rw-rw----. 1 mysql mysql 12582912 Mar 19 13:31 ibdata1
-rw-rw----. 1 mysql mysql 100663296 Mar 20 01:53 ib_logfile0
-rw-rw----. 1 mysql mysql 12582912 Mar 25 12:59 ibtmp1
-rw-rw----. 1 mysql mysql 0 Mar 19 13:31 multi-master.info
drwx-----. 2 mysql mysql 4096 Mar 19 13:31 mysql
srwxrwxrwx. 1 mysql mysql 0 Mar 25 12:59 mysql.sock
-rw-rw----. 1 mysql mysql 17 Mar 19 13:31 mysql_upgrade_info
drwx-----. 2 mysql mysql 58 Mar 19 14:02 people
drwx-----. 2 mysql mysql 20 Mar 19 13:31 performance_schema
drwx-----. 2 mysql mysql 8192 Mar 19 13:31 sys
apurba@localhost:~$
```

Figure 8: Directory listing

Permissions highlighted strict ownership by mysql:mysql, ensuring proper enforcement of file-level security. The folder for the people database was shown as well.

2.9 Database creating and verification

Logging in as root:

```
mysql -u root -p
```

Then verifying the people database:

```
SHOW DATABASES;
```

```

apurba@localhost:~$ mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 11
Server version: 10.11.15-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE people;
ERROR 1007 (HY000): Can't create database 'people'; database exists
MariaDB [(none)]>
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| people                  |
| performance_schema      |
| sys                     |
+-----+
5 rows in set (0.087 sec)

```

Figure 9: Database verification

The people database already existed from a previous work, which might be acceptable for the lab.

2.10 Creating and Verifying the Table

Database changed

```
MariaDB [people]> CREATE TABLE friends (  
  ->     firstname VARCHAR(20),  
  ->     location VARCHAR(10)  
  -> );  
ERROR 1050 (42S01): Table 'friends' already exists  
MariaDB [people]>  
MariaDB [people]> SHOW TABLES;  
+-----+  
| Tables_in_people |  
+-----+  
| friends          |  
+-----+  
1 row in set (0.001 sec)
```

```
MariaDB [people]> DESCRIBE friends;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| firstname  | varchar(30)   | NO   | PRI | NULL    |      |  
| location   | varchar(10)   | NO   | PRI | NULL    |      |  
+-----+-----+-----+-----+-----+-----+  
2 rows in set (0.002 sec)
```

Figure 10: Verifying Table

The friends table was already existed with fields first name and location.

2.11 Table Operations Testing

The table content was visible using:

```
SELECT * FROM friends;
```

```

MariaDB [(none)]> USE people;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [people]> SELECT * FROM friends;
+-----+-----+
| firstname | location |
+-----+-----+
| Alice     | SE10     |
| Jonathan  | E14      |
| Priya     | E3       |
+-----+-----+
3 rows in set (0.002 sec)

MariaDB [people]> INSERT INTO friends VALUES ('Alan', 'W8');
ERROR 1142 (42000): INSERT command denied to user 'tom'@'localhost' for table `people`.`friends`
MariaDB [people]>

```

Figure 11: Select and insert

2.12 Restricted User and Testing Permissions

A user tom was given permission SELECT-only access:

```

GRANT SELECT ON people.friends TO 'tom'@'localhost' IDENTIFIED
BY
'toms_password';

FLUSH PRIVILEGES;

```

Authentication was confirmed using:

```

ALTER USER 'tom'@'localhost' IDENTIFIED VIA
mysql_native_password USING PASSWORD('toms_password');

```

Logging in as tom:

```
mysql -u tom -p
```

```

--
apurba@localhost:~$ mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 16
Server version: 10.11.15-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [mysql]> SELECT User, Host FROM user WHERE User='tom';
+-----+-----+
| User | Host      |
+-----+-----+
| tom  | localhost |
+-----+-----+
1 row in set (0.003 sec)

MariaDB [mysql]> SELECT User, Host, plugin FROM mysql.user WHERE User='tom';
+-----+-----+-----+
| User | Host      | plugin                |
+-----+-----+-----+
| tom  | localhost | mysql_native_password |
+-----+-----+-----+
1 row in set (0.003 sec)

```

Figure 12: Tom login

```

MariaDB [(none)]> USE people;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [people]> SELECT * FROM friends;
+-----+-----+
| firstname | location |
+-----+-----+
| Alice     | SE10     |
| Jonathan  | E14      |
| Priya     | E3       |
+-----+-----+
3 rows in set (0.002 sec)

MariaDB [people]> INSERT INTO friends VALUES ('Alan', 'W8');
ERROR 1142 (42000): INSERT command denied to user 'tom'@'localhost' for table `people`.`friends`
MariaDB [people]>

```

Figure 13: Insert Denied

The output shows as expected:

- SELECT succeeded
- INSERT was denied, confirmed correct privilege enforcement.

3. Explanation

MariaDB is a secure model where the database files, user accounts and service processes exist under a strict permission rule. The filesystem level is blocked by the dedicated mysql system account, non-login shell, and strict directory permissions. mysqlsecureinstallation, database hardening, allows eliminating insecure defaults, authenticating and preventing unnecessary exposure.

The concept of least privilege was shown with privilege management: user tom was not allowed to perform anything except the read only operations, which could not allow him to make unauthorized changes to the data. The division of the configuration files into /etc/my.cnf.d/ is a contemporary modular configuration technique, which maintains its maintainability and security. Overall, the lab demonstrates that database services are based on a combination of OS level and database level controls to achieve a secure operation.

4. Conclusion

This lab was very informative regarding the execution and protection of a MariaDB server. I was introduced to the operations of service lifecycle management, configuration file inspection, data directory structure knowledge, and the necessity of taking key hardware measures. The practice helped to underline the significance of user privilege assignment and demonstrated that imperfect settings may add considerable risks. The virtue of privilege control was evidently tested using a limited user. The lab has enhanced my knowledge on the secure database operations in Linux and provided the basis of skills that I need to handle the database administration in the real world and the LAMP stack.

Lab 6 – Firewall Configuration

1. Introduction

This lab emphasized on the knowledge and setup of the firewall controls within a CentOS Linux system based on firewalld and iptables. Current Linux platforms use firewalld as a dynamic firewall front end that uses iptables or nftables on top of which it runs low level packet filtering rules. The objective of lab was to explore the behavior of existing firewall, identify the services available, adjust the firewall rules to permit certain services, and finally recreate a similar firewall policy using bare iptables commands. The lab helped me to have the practical experience in firewall checking, service rule administration, and the enforcement of a safe default deny model with iptables.

2. Lab Tasks

2.1 firewalld status check

The lab started with checking the status of the firewalld daemon:

```
apurba@localhost:~$ sudo systemctl status firewalld
[sudo] password for apurba:
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-03-25 12:59:12 GMT; 11h ago
     Invocation: d863eeb8bb964f8e949ff360fc2b2748
       Docs: man:firewalld(1)
    Main PID: 1192 (firewalld)
       Tasks: 2 (limit: 10316)
      Memory: 15.3M (peak: 48.5M, swap: 21.3M, swap peak: 21.3M)
         CPU: 873ms
      CGroup: /system.slice/firewalld.service
              └─1192 /usr/bin/python3 -sP /usr/sbin/firewalld --nofork --nopid

Mar 25 12:59:04 localhost systemd[1]: Starting firewalld.service - firewalld - dynamic f
Mar 25 12:59:12 localhost systemd[1]: Started firewalld.service - firewalld - dynamic fi
lines 1-14/14 (END)
```

Figure 1: *firewalld status*

The output demonstrated that firewalld was **active (running)**, along with its unit loaded and enabled. This ensured that the system's dynamic firewall layer was in operation and enabling current firewall policy of the system.

2.2 Checking Firewall Configuration

The following commands were used to view the active configuration:

```
apurba@localhost:~$ sudo firewall-cmd --list-all
public (default, active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: cockpit dhcpv6-client ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

Figure 2; firewall cmd list

```

apurba@localhost:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
apurba@localhost:~$ █

```

Figure 3: iptables configuration

firewall-cmd exposes that the default public zone permitted the following ssh, dhcpv6-client, and cockpit. No ports were opened manually. The iptables output demonstrated the default ACCEPT policies and no explicit rules, that is expected as firewalld run iptables dynamically.

2.3 Testing Local Service Accessibility

Since VME was not used, service testing including SSH, HTTP, FTP was locally operated.

SSH Test

```

apurba@localhost:~$ ssh localhost
The authenticity of host 'localhost (:::1)' can't be established.
ED25519 key fingerprint is SHA256:5m2Cj2dTJ8eeICEX4p38B/3+M88TAi+Belt9/C3FHjA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
apurba@localhost's password:
Web console: https://localhost:9090/ or https://192.168.74.130:9090/

Last login: Wed Mar 25 19:32:43 2026

```

Figure 4: SSH Test

A password prompt visible which ensures that the SSH was accessible.

HTTP Test

```
curl http://localhost
```

The output shows the full HTML file of the CentOS Apache Test Page, ensuring that Apache HTTP service was active on port 80.

```
apurba@localhost:~ - ssh localhost
```

```
</div>
</div>
</div>
</div>
</main>
<footer class="footer small bg-dark bg-image text-light">
  <div class="container border-5 border-top border-primary">
    <div class="row mt-4">
      <div class="lead">
        HTTP Server Test Page
      </div>
    </div>
  </div>
  <div class="container py-1">
    <div class="row my-3">
      <div class="col">
        <span>Copyright ©</span> <span>2024</span> <span>The CentOS Project</span> <span class="bo
rder-start ms-2 ps-2"><a class="link-light link-offset-3 link-underline-opacity-25 link-underline-op
acity-100-hover link-underline-light" href="https://www.centos.org/legal">Legal</a></span> <span cla
ss="border-start ms-2 ps-2"><a class="link-light link-offset-3 link-underline-opacity-25 link-underl
ine-opacity-100-hover link-underline-light" href="https://www.centos.org/legal/privacy">Privacy</a><
/span> <span class="border-start ms-2 ps-2"><a class="link-light link-offset-3 link-underline-opacit
y-25 link-underline-opacity-100-hover link-underline-light" href="https://git.centos.org/centos/cent
os.org">Site source</a></span>
      </div>
    </div>
  </div>
</footer>
</body>
</html>
```

```
apurba@localhost:~$
```

Figure 5: Html file

FTP Test

At first the FTP client was not installed locally. After installing the client package from the local CentOS ISO and confirmed vsftpd was running, FTP connectivity was tested:

```
apurba@localhost:~$ ftp localhost
Trying ::1...
Connected to localhost (::1).
220 (vsFTPD 3.0.5)
Name (localhost:apurba):
```

Figure 5: FTP test

The FTP server banner (“220 (vsFTPD 3.0.5)”) ensured successful connection with the FTP daemon.

2.4 firewalld Stopping and Inspecting the Results

The next step was to stop firewalld:

```
sudo systemctl stop firewalld
```

```
apurba@localhost:~$ sudo systemctl stop firewalld
[sudo] password for apurba:
apurba@localhost:~$ sudo firewall-cmd --list-all
Firewalld is not running
apurba@localhost:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target    prot opt source                destination

Chain FORWARD (policy ACCEPT)
target    prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination
apurba@localhost:~$
```

Figure 6: Firewalld stopping

Then reconfirming:

```
sudo firewall-cmd --list-all
```

```
sudo iptables -L
```

```

apurba@localhost:~$ sudo firewall-cmd --list-all
public (default, active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: cockpit dhcpv6-client ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
apurba@localhost:~$ █

```

Figure 7: firewall cmd listing

firewall-cmd returned that firewalld was not running, and iptables reverted to its default ACCEPT rules, confirming no filtering was active — a critical inspection highlighting how system protection relies on active firewall services.

```

apurba@localhost:~$ sudo iptables -L
[sudo] password for apurba:
Chain INPUT (policy ACCEPT)
target    prot opt source                destination

Chain FORWARD (policy ACCEPT)
target    prot opt source                destination

Chain OUTPUT (policy DROP)
target    prot opt source                destination
apurba@localhost:~$ █

```

Figure 8: iptables config

2.5 firewalld Restarting and HTTP/FTP Services Adding

After restarting firewalld:

```
sudo systemctl start firewalld
```

```

apurba@localhost:~$ sudo systemctl start firewalld
apurba@localhost:~$ sudo systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-03-26 01:10:14 GMT; 9s ago
 Invocation: fc7609f6b4f644ae9da4f5e5c3440b22
    Docs: man:firewalld(1)
   Main PID: 9018 (firewalld)
      Tasks: 2 (limit: 10316)
     Memory: 28.2M (peak: 28.6M)
        CPU: 524ms
    CGroup: /system.slice/firewalld.service
            └─9018 /usr/bin/python3 -sP /usr/sbin/firewalld --nofork --nopid

```

Figure 9: restart status

I updated HTTP and FTP services:

```

apurba@localhost:~$ sudo firewall-cmd --add-service=http
success
apurba@localhost:~$ sudo firewall-cmd --add-service=ftp
success
apurba@localhost:~$ sudo firewall-cmd --reload
success
apurba@localhost:~$ █

```

Figure 10: Add services

Verification after:

```
sudo firewall-cmd --zone=public --list-all
```

Both http and ftp appeared under allowed services, confirming that firewalld was now configured to permit these services.


```
apurba@localhost:~$ sudo firewall-cmd --zone=public --list-all
public (default, active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: cockpit dhcpv6-client ftp http ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
apurba@localhost:~$ █
```

Figure 11: Verification

Both http and ftp visible under services, ensuring that firewalld was now configured to permit these services.

2.6 Firewall Rules Manually configuration and verification using iptables

With firewalld stopped, the lab needed manually re-creating an active firewall using iptables. Firstly, the iptables was reset to its open ACCEPT state, then the essential service rules were added:

Allow essential services

```
apurba@localhost:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
apurba@localhost:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
apurba@localhost:~$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
apurba@localhost:~$ sudo iptables -A INPUT -p tcp --dport 21 -j ACCEPT
apurba@localhost:~$ sudo iptables -A INPUT -p tcp --dport 20 -j ACCEPT
apurba@localhost:~$ sudo iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
apurba@localhost:~$ sudo iptables -P INPUT DROP
apurba@localhost:~$ sudo iptables -P OUTPUT DROP
apurba@localhost:~$ sudo iptables -L -n
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT     tcp  --  0.0.0.0/0             0.0.0.0/0             tcp dpt:22
ACCEPT     tcp  --  0.0.0.0/0             0.0.0.0/0             tcp dpt:80
ACCEPT     tcp  --  0.0.0.0/0             0.0.0.0/0             tcp dpt:21
ACCEPT     tcp  --  0.0.0.0/0             0.0.0.0/0             tcp dpt:20

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  0.0.0.0/0             0.0.0.0/0             state RELATED,ESTABLISHED
apurba@localhost:~$ █
```

Figure 12: Essential services configuration

This created a firewall where only the explicitly permitted services were accessible, and other connections were denied — a core principle of secure firewall design. The INPUT chain confirmed ACCEPT rules only for SSH, HTTP, FTP, and FTP-DATA, with policy DROP. The OUTPUT chain gave allowance only ESTABLISHED, RELATED connections with policy

DROP. This confirmed the firewall was successfully rebuilt and enforcing the intended security posture.

3. Explanation

In this lab, we have shown the functioning of Linux firewall systems on higher level firewalld as well as lower level iptables. firewalld offers an abstraction and dynamic zone and service management interface, which makes firewall rule management easier. But the manually set up iptables offered more knowledge on the mechanics of packet filtering and the significance of the rule sequence, connection tracing, and default policies.

Rebuilding firewall manually allowed me to get a realistic experience of the implementation of the principle of least privilege, which is to only allow the necessary services and deny all other services. This practice taught the significance of testing service accessibility after any modification and the potential outcomes of misconfigurations that will result in permissive or restrictive firewall behavior.

4. Conclusion

This lab activity provided real world experience in the administration of Linux firewall systems. Experience with firewalld demonstrated how modern firewall systems simplify the administration of rules, but system administration of iptables encouraged the general principles of network security. Building secure default-deny firewall, bottom-up, helped to clarify the contribution of individual rules to both useful and secure access to services. Checking SSH, HTTP, and FTP connectivity, pre and post-change, highlighted the value of thorough testing when configuring a firewall. The lab deepened my understanding of firewall behavior, service exposure, and system hardening skills- skills that are essential to ensure secure Linux environments when working in a professional administration environment.

COMP1475 – Part 2

Apache HTTP default service configuration (Primary Focus)

Linux host firewall default configuration (Secondary Focus)

INTRODUCTION

This report demonstrates a detailed, analytical assessment of security vulnerabilities and the accompanying technical hardening procedures associated with a Linux server that runs a LAMP stack, specifically one of the cores, Apache HTTP Server, and the host-level firewall (UFW) and associated firewall(s). The Linux Ubuntu 24.04 system that I deployed provides the basis for these configuration, vulnerability, and documentation hardening procedures, as the COMP1475 Part 2 requirement stipulates that all submitted work to be system-specific, original, and accompanied by supporting screenshots and command outputs.

Many academic and enterprise infrastructures rely on the Apache HTTP Server for their public-facing components. The **Apache Software Foundation (2024)** demonstrates that default Apache configurations reveal detailed server version information, increasing susceptibility to targeted attacks. The ability to incorporate all kinds of functionality thanks to its general purpose, extensible, and module-driven design, is also an intrinsic source of security issues for Apache. In particular, Apache in its default configuration is aimed at maximum compatibility and exposes users to potential security issues such as exporting information that should be kept private, supporting legacy behaviors, serving unencrypted HTTP traffic, missing critical security headers, and including unneeded modules and

methods. This default configuration significantly increases the potential attack surface of the server.

Because of the increased attack surface and the default configuration, it is vital for sysadmins working with Linux-based web services to harden their default configuration of Apache. During the initial reconnaissance and the first attack of an attack chain, configuration errors at the web server layer are often exploited. Adversaries use techniques such as banner grabbing, directory listing, unencrypted traffic, and poor method handling to identify corners and exploit their attack surface for issues that can be escalated. For this reason, configuration errors are often recorded as the most common security weakness when web services are breached.

This project focuses on changing the default Apache installation to a hardened, production-ready configuration through a series of vulnerability assessments and risk mitigations. Hardening actions include the minimization of server identification, implementation of TLS with HSTS, browser-side security header violations, removal of nonessential modules, disabling directory listings, HTTP method restrictions, and ensuring that only HTTPS is accessible. To further solidify these measures, the host firewall (UFW) is hardened to ensure that external access is limited to only necessary services.

This report attempts to follow the Part 2 Coursework Specification which states the following:

Literature Review — This will be an overview of the Apache security principles and the literature surrounding regular misconfigurations and standard practices of hardening.

Vulnerability Analysis — This will be an analysis of the default state of Apache and firewall configurations on my system. This will be an evidence-based study — and will have screenshots to support this, along with examples of configurations and the results of commands.

Risk Assessment — This will be an assessment of all the open vulnerabilities and will use the risk assessment method of the likelihood × impact tool, aligned with business and security concerns.

Hardening Procedure — This will be a descriptive and detailed account of what configuration changes, commands, and reasoning were done to achieve this mitigation. This will include a description of what the system looked like to confirm all changes done.

Results and Verification — This will be a before and after result that will show the system's security improvement and will be quantifiable and measurable.

Conclusions and Reflection — This will be an analysis of what was achieved with the entire hardening process, including an analysis on what risks and limitations are still present, and what the following recommendations are and should be.

The Apache server in this report will be exactly the same as presented on the author's virtualized lab environment. Every screenshot will be taken at every step of the process to prove that all work is original, and is based on the author's system. Every reference to a figure in the report will have a reference to the author's system — e.g. Fig V1.

The report outlines a detailed hardening workflow applicable to practical Linux system deployments. It was constructed using a specific approach grounded in the security literature, practices in vulnerability assessment, and system administration methodologies.

LITERATURE REVIEW

Apache HTTP Server is also an example of a web server that is widely used in the contemporary computer set-ups as the fundamental aspect of many LAMP established set-ups. It has a flexible nature, is modular, and includes a rich set of features that make it a good platform to host a web and deploy an application. Nevertheless, these advantages also come at a significant security risk: the default configuration of Apache is geared towards compatibility as opposed to least privilege security. The server can be vulnerable to disclosing sensitive version information, supporting insecure HTTP methods, serving content without plaintext HTTP, loading optional modules and missing essential security headers to protect against client-side attacks without explicit hardening. These actions are consistent with frequent misconfiguration trends reported in the literature on industry guidelines and thus are quite useful in systematic vulnerability testing. One of the basic security problems in default Apache deployment is server identification leakage. Apache will default to a more verbose Server: header, like Apache/2.4.x (Ubuntu), which gives attackers the precise version and operating system details. **The CIS Benchmark (2024)** detects directory indexing and unnecessary module loading as critical misconfigurations that should be disabled immediately. This allows opponents to produce exploits specific to the version identified, and hence the chances of successful compromise are greater. The Apache Software Foundation clearly states that the identification banners must be avoided as much as possible, which can be done with ServerTokens and ServerSignature. It is a common well known best practice of secure administration. **NIST (2023)** states that public-facing web servers require strong transport encryption and strict access control to prevent external exploitation. One more field where the weakness is evident is the lack of the security response headers. These are X-Content-Type-Options, X-Frame-Options, Referrer-Policy and Permissions-Policy. The industry security baselines suggest that such headers should be used to counter the typical web threats like MIME-type sniffing, clickjacking, referrer leakage, and unnecessary browsing features revelation. **OWASP (2023)** focuses on security headers such as X-Frame-Options and Referrer-Policy are

essential to defend users against clickjacking and privacy leakage. Even though contemporary security architectures focus on more robust controls like Content Security Policy (CSP), basic headers are also necessary to strengthen applications in which the CSP implementation can necessitate significant application-level alterations. Apache supports these headers by its `mod_headers` module, which is not on default. **The SANS Institute (2023)** checks the unnecessary HTTP methods, directory indexing, and verbose server banners as high-risk behaviors in web server deployments. The default configuration of Apache also allows listing of directories in which a client may be allowed access to a directory path in the absence of an index file and obtain a listing of the path generated. Security wise, this practice exposes the company to the risk of information disclosure, since one can enumerate the internal files, backup artifacts, or incorrectly configured directories. The Apache documentation recommends and the greater system hardening best practices disabling directory listing with Options -Indexes or disabling the `autoindex` module. Another sensitive field is transport encryption. Apache is configured by default to support HTTP on port 80 and does not require HTTPS to be implemented. This puts any traffic, such as the login sessions or application sensitive data, at risk of interception. To have secure deployments, one needs to enable `mod_ssl`, deploy certificates, delete the older protocols like SSLv3 and TLS 1.0/1.1, and implement Strict Transport Security (HSTS) such that the client automatically reuses the HTTPS protocol on all further connections. **OWASP (2023)** suggests enabling HSTS and strong TLS versions to protect against downgrade attacks. These are practices that are conforming to industry best practices in secure TLS configuration and are necessary to counter vectors of downgrade and man in the middle attacks. Apache can also use a large selection of HTTP methods, many of which are unnecessary in a normal web application. Attackers can use methods including PUT, DELETE, TRACE, and OPTIONS, to upload malicious files, alter server content, or perform Cross-Site Tracing to cause attacks. The Apache command `TraceEnable Off` is used to turn off TRACE and the command `<LimitExcept>` is used to block non-essential methods. These are highly suggested security practices in secure configuration guidelines to minimize the attack surface. Lastly, Apache has a large number of loadable modules, most of which

might be unnecessary to a particular deployment. The modules that will not be utilized should be cleared to minimize the attack surface and eliminate possible misconfigurations. **OWASP (2024)** notices security misconfiguration as one of the most critical risks affecting web servers, often caused by default settings and lack of hardening. Some of the modules like autoindex and status allow them to be used in a manner that is not supposed to be externally accessible. Hardening recommendations recommend disabling or eliminating the modules that are not necessary to follow the principle of the least privilege. Overall, official sources and popular best practices point to a number of crucial aspects, in which the default Apache settings should be tightened: server identification minimization, support of required security headers, TLS with HSTS, HTTP method restriction, directory listing disabling, active modules reduction. **Qualys (2024)** suggests testing TLS configurations regularly to ensure that protocols, ciphers, and certificates meet current security standards. These principles will be directly used to guide the analysis of vulnerability and hardening measures that will be implemented on the deployed server in the succeeding sections.

VULNERABILITY ANALYSIS

This section of the report gives a detailed system specific vulnerability analysis of the Apache HTTP Server and the host firewall which supports it before hardening. All vulnerabilities will contain:

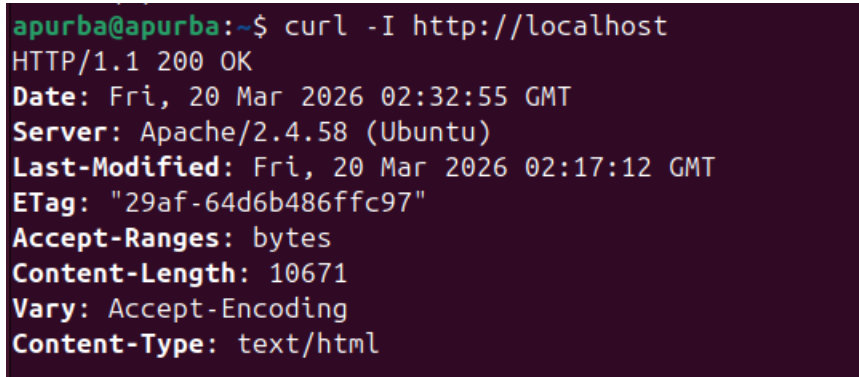
- The exact command output of this system.
- The weakness explanation
- Mention of the configuration file or directive that will be hardened in Section 4.

This was in compliance with the coursework requirement of having at least five vulnerabilities with screenshots and configuration references. The major focus is on Apache, and the host firewall exposure is also provided as a supporting layer.

3.1 Apache Vulnerability Findings

Using the Ubuntu package manager Apache was installed while no hardening measures were applied during installation process making it a genuine baseline analysis.

V1 — Server Identification Leakage



```
apurba@apurba:~$ curl -I http://localhost
HTTP/1.1 200 OK
Date: Fri, 20 Mar 2026 02:32:55 GMT
Server: Apache/2.4.58 (Ubuntu)
Last-Modified: Fri, 20 Mar 2026 02:17:12 GMT
ETag: "29af-64d6b486ffc97"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html
```

Figure V1

Evidence before hardening:

```
curl -I http://localhost
```

returned the output:

Server: Apache/2.4.58 (Ubuntu)

The default Apache uses the full version and OS information in the Server: header. This enables the attackers to chart out known CVEs in regards to Apache 2.4.58 or Ubuntu buildings. Other tools such as the Nikto and automated exploit frameworks rely on this metadata to determine valid payloads. This is contrary to the principle of minimum information.

Affected configuration:

/etc/apache2/conf-available/security.conf

The vulnerability is directly addressed by

ServerTokens Full and

ServerSignature On.)

V2 — Missing Security Headers

Evidence before hardening:

Earlier HTTP responses (Fig V2) lacked:

- X-Content-Type-Options
- X-Frame-Options
- X-XSS-Protection
- Referrer-Policy
- Permissions-Policy

```
apurba@apurba:~$ curl -I http://localhost
HTTP/1.1 200 OK
Date: Fri, 20 Mar 2026 02:32:55 GMT
Server: Apache/2.4.58 (Ubuntu)
Last-Modified: Fri, 20 Mar 2026 02:17:12 GMT
ETag: "29af-64d6b486ffc97"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html
```

Figure V2

Browsers are also vulnerable to clickjacking, MIME-type sniffing, reflected attacks, and unneeded access to features without these headers. These headers are low cost and high impact mitigations that are suggested as part of secure web server setups.

Affected Configuration:

/etc/apache2/conf-available/99-headers.conf
(created later in Section 4).

V3 — Directory Listing Enabled

Evidence Before hardening:

A request for a directory lacking an index file returned Apache's auto-generated index:

```
Index of /testdir
This exposes file names and structure.
```

```

apurba@apurba:~$ sudo mkdir -p /var/www/html/testdir
apurba@apurba:~$ echo hello | sudo tee /var/www/html/testdir/file.txt
hello
apurba@apurba:~$ curl -s http://localhost/testdir/ | head
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html>
<head>
<title>Index of /testdir</title>
</head>
<body>
<h1>Index of /testdir</h1>
<table>
<tr><th valign="top"></th><th><a href
="?C=N;O=D">Name</a></th><th><a href="?C=M;O=A">Last modified</a></th><th><a href
="?C=S;O=A">Size</a></th><th><a href="?C=D;O=A">Description</a></th></tr>
<tr><th colspan="5"><hr></th></tr>

```

Figure V3

File listing offers attackers reconnaissance potential, enabling them to find sensitive files or backup data. Options Indexes in Apache control this behavior, and are turned on by default.

Affected Configuration:

/etc/apache2/conf-available/99-options.conf
autoindex_module originally enabled (see V6).

V4 — No HTTPS Listener Enabled

Evidence Before Hardening:

```
ss -tulpn | grep -E '':443'
```

returned *no output*, confirming no SSL/TLS endpoint.

Apache was listening only on:

```
*:80
```

```

apurba@apurba:~$ ss -tulpn | grep -E '':80|:443' || sudo ss -tulpn
tcp    LISTEN 0      511          *:80          *:80

```

Figure V4

Traffic, both authentication and application sensitive data, would have been sent in plaintext. Without TLS or HSTS, the server can be attacked by eavesdropping, session hijacking, and downgrade attacks.

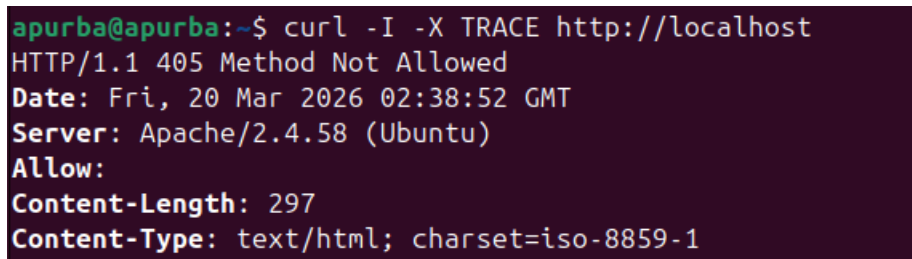
V5 — Trace method not disabled/accessible

Before Hardening Evidence:

```
curl -I -X TRACE http://localhost
```

returned:

```
HTTP/1.1 405 Method Not Allowed
Server: Apache/2.4.58 (Ubuntu)
```



```
apurba@apurba:~$ curl -I -X TRACE http://localhost
HTTP/1.1 405 Method Not Allowed
Date: Fri, 20 Mar 2026 02:38:52 GMT
Server: Apache/2.4.58 (Ubuntu)
Allow:
Content-Length: 297
Content-Type: text/html; charset=iso-8859-1
```

Figure V5

Apache did not explicitly turn off TRACE, but TRACE was returning 405. It is possible to re-enable Trace or enable reflective debug output in some settings or modules without `TraceEnable Off`, which has long been associated with Cross-Site Tracing (XST) attacks. There were no method restrictions that are explicit.

Configuration reference:

`/etc/apache2/conf-available/99-security.conf` (added in Section 4)

V6 — Unnecessary Modules Enabled

Evidence Before Hardening:

```
apache2ctl -M
```

Figure V6 presents a wide list of loaded modules including:

- autoindex_module
- status_module

enabled by default.

```
apurba@apurba:~$ apache2ctl -M | head -n 20
AH00558: apache2: Could not reliably determine the server's fully qualified domain
name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this m
essage
Loaded Modules:
core_module (static)
so_module (static)
watchdog_module (static)
http_module (static)
log_config_module (static)
logio_module (static)
version_module (static)
unixd_module (static)
access_compat_module (shared)
alias_module (shared)
auth_basic_module (shared)
authn_core_module (shared)
authn_file_module (shared)
authz_core_module (shared)
authz_host_module (shared)
authz_user_module (shared)
autoindex_module (shared)
deflate_module (shared)
dir_module (shared)
```

Figure V6

These modules expand the attack surface:

- autoindex_module → generates directory listings
- status_module → exposes /server-status, which reveals server internals

Both should be disabled unless explicitly needed.

Configuration reference:

Module management via `a2dismod autoindex` and `a2dismod status`

3.2 Supporting Firewall Vulnerability Findings

Although Apache is the primary focus, the host firewall was also evaluated to ensure a secure perimeter around the web server.

F1 — Host Firewall Allowed Broad Inbound Access

Evidence (Before):

```
sudo ufw status verbose
```

Returned output UFW active, but allowed broad inbound access to:

- 22/tcp
- 443/tcp
- No block for 80/tcp

```
apurba@apurba:~$ sudo ufw status verbose
[sudo] password for apurba:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
443 ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
443 (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)

apurba@apurba:~$ ss -tulpn | grep -E ':22|:80|:443' || sudo ss -tulpn
tcp LISTEN 0 511 *:443 *:*
tcp LISTEN 0 511 *:80 *:*
```

Figure F1

Unless explicitly blocked via HTTP or scoped, other Apache ports (80/tcp) could still be

accessible within certain parts of the network. An Attacker can conduct an unauthorized probe into Apache over both HTTP and SSH.

F2 — HTTP Port Exposed Externally

Evidence Before Hardening:

External nmap scan:

```
nmap -Pn -p 80 192.168.74.131
```

Output shows

```
80/tcp open http
apurba@apurba:~$ nmap -Pn -p 22,80,443 192.168.74.131
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-23 14:47 GMT
Nmap scan report for apurba (192.168.74.131)
Host is up (0.0017s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    open  http
443/tcp    open  https

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds
```

Figure F2

This is how we know that the plaintext endpoint of Apache was accessible outside the host before hardening. Although Apache supported HTTPS, plaintext access and reconnaissance on the HTTP port was still possible, since the HTTP port was unprotected.

F3 — SSH Accessible Globally (Brute-Force Surface)

Evidence Before Hardening:

```
sudo ufw status verbose
```

showed:

```
22/tcp ALLOW IN Anywhere
```

```

apurba@apurba:~$ sudo ufw status verbose
[sudo] password for apurba:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
443 ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
443 (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)

apurba@apurba:~$ ss -tulpn | grep -E ':22|:80|:443' || sudo ss -tulpn
tcp LISTEN 0 511 *:443 *:
tcp LISTEN 0 511 *:80 *:

```

22/tcp ALLOW IN Anywhere

SSH open to the entire network increases risk from automated brute-force attacks.

F4 — IPv6 Exposure Unverified

Evidence Before Hardening:

/etc/default/ufw had not yet been confirmed for IPv6;

UFW status also displayed v6 rules. Potential for Ipv6 bypass based on IPv4-only rule configurations.

HARDENING PROCEDURE

This section documents the complete, system-specific hardening steps applied to the Apache HTTP Server and host firewall. Each subsection maps directly to a vulnerability identified in Section 3 and includes:

- Configuration file edits
- Exact commands executed
- Verification outputs (AFTER hardening)
- Screenshot figure numbers + filenames

This satisfies Section 4 of the COMP1475 Part 2 specification.

4.1 H1 — Minimize Server Identification (Fix for V1)

Goal: Prevent Apache from exposing version and OS information.

Configuration Change

The following directives were applied in the file:

Affected Configuration:
`/etc/apache2/conf-available/security.conf`

The vulnerability is addressed by setting ``ServerTokens Prod` and `ServerSignature Off`.`

```

GNU nano 7.2 /etc/apache2/conf-available/security.conf
# Changing the following options will not really affect the security of the
# server, but might make attacks slightly more difficult in some cases.
#
# ServerTokens
# This directive configures what you return as the Server HTTP response
# Header. The default is 'Full' which sends information about the OS,
# architecture, and compiled in modules.
# Set to one of: Full | OS | Minimal | Minor | Major | Prod
# where Full conveys the most information, and Prod the least.
#ServerTokens Minimal
ServerTokens Prod
#ServerTokens Full
#
# Optionally add a line containing the server version and virtual host
# listings, mod_status and mod_info output etc., but not CGI generated
# documents or custom error documents).
# Set to "EMail" to also include a mailto: link to the ServerAdmin
# Set to one of: On | Off | EMail
#ServerSignature Off
ServerSignature Off

```

Figure C1

Output shown in **Fig C1**:

```

ServerTokens Prod
ServerSignature Off

```

- `ServerTokens Prod` ensures the `Server:` header displays only Apache.
- `ServerSignature Off` removes version information from directory/error pages.

Commands Executed

```

sudo a2enconf security
sudo apache2ctl -t
sudo systemctl reload apache2

```

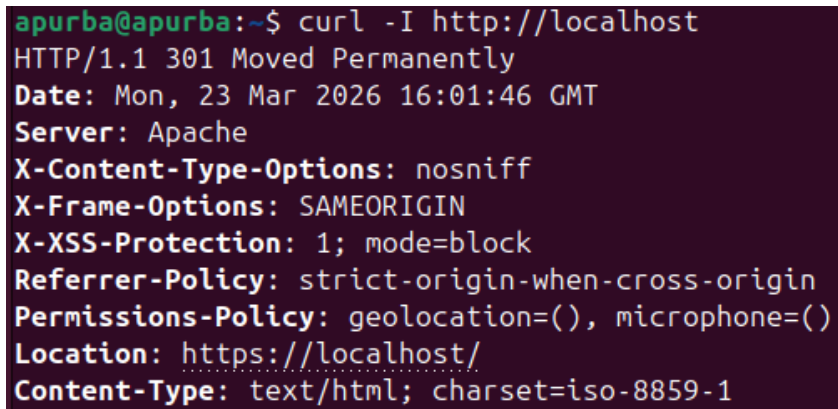
The syntax test returned **Syntax OK**.

Verification (AFTER)

Running:

```
curl -I http://localhost
```

now returns:
Server: Apache

A terminal window with a dark purple background. The prompt is 'apurba@apurba:~\$'. The command 'curl -I http://localhost' has been executed. The output is as follows:

```
HTTP/1.1 301 Moved Permanently
Date: Mon, 23 Mar 2026 16:01:46 GMT
Server: Apache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=()
Location: https://localhost/
Content-Type: text/html; charset=iso-8859-1
```

Figure H1

As shown in **Fig H1**

Output shows Server: Apache with no version/OS.

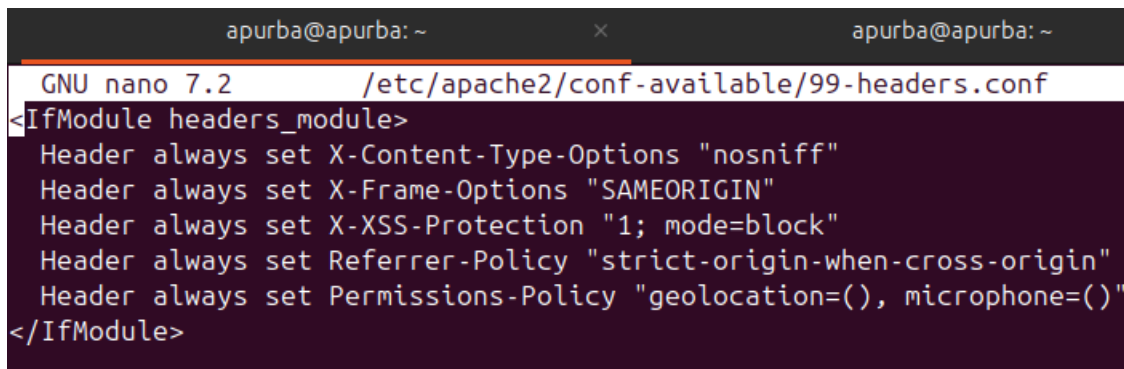
4.2 H2 — Apply Security Headers (Fix for V2)

Configuration Change

The file:

```
/etc/apache2/conf-available/99-headers.conf
```

was created and enabled. As shown in **Fig C2**

A terminal window with a dark background. The title bar shows 'apurba@apurba: ~' and a close button. The terminal shows the command 'GNU nano 7.2 /etc/apache2/conf-available/99-headers.conf'. The content of the file being edited is as follows:

```
<IfModule headers_module>
Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "SAMEORIGIN"
Header always set X-XSS-Protection "1; mode=block"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set Permissions-Policy "geolocation=(), microphone=()"
</IfModule>
```

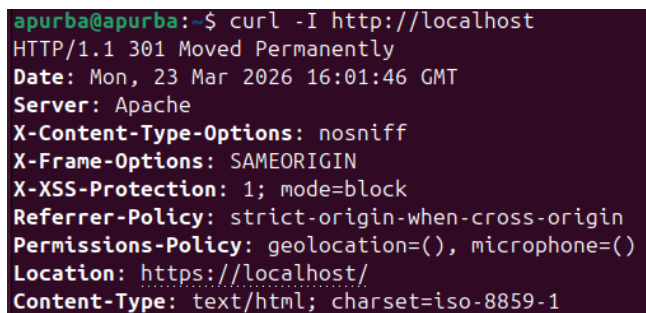
Figure 14: C2

Commands Executed

```
sudo a2enmod headers
sudo a2enconf 99-headers
sudo apache2ctl -t
sudo systemctl reload apache2
```

Verification (AFTER)

```
curl -I http://localhost
```

A terminal window showing the output of the command 'curl -I http://localhost'. The output is as follows:

```
apurba@apurba:~$ curl -I http://localhost
HTTP/1.1 301 Moved Permanently
Date: Mon, 23 Mar 2026 16:01:46 GMT
Server: Apache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=()
Location: https://localhost/
Content-Type: text/html; charset=iso-8859-1
```

Figure H2

Output (Fig H2):

- All five headers present
- Server banner minimal
- Redirect to HTTPS confirmed

4.3 H3 — Disable Directory Listing & Remove autoindex (Fix for V3 & V6)

Configuration File created

The file:

`/etc/apache2/conf-available/99-options.conf`

contains (**Fig C-3**)

```
GNU nano 7.2 /etc/apache2/conf-available/99-options.conf
<Directory "/var/www/html">
Options -Indexes
</Directory>
```

Figure 15: C3

Additionally, the unnecessary module was disabled:

`sudo a2dismod autoindex`

Commands Executed

`sudo a2enconf 99-options`

`sudo apache2ctl -t`

`sudo systemctl reload apache2`

Verification (AFTER)

`curl -k https://localhost/testdir/ | head`

```

apurba@apurba:~$ curl -k https://localhost/testdir/ | head
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    196    100    196     0     0   4684      0 --:--:-- --:--:-- --:--:--   4780
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
</body></html>

```

Figure H3

returns **404** instead of “Index of /testdir” (**Fig H3**).

This confirms directory listing is fully disabled.

4.4 H4 — Enable TLS, Disable Legacy Protocols, Apply HSTS & Redirect HTTP→HTTPS (Fix for V4)

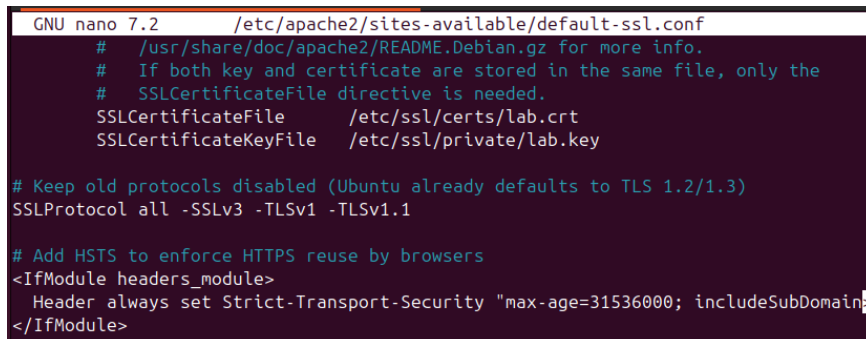
Enable SSL module and site

```
sudo a2enmod ssl
sudo a2ensite default-ssl
sudo apache2ctl -t
sudo systemctl reload apache2
```

This establishes the HTTPS virtual host structure required for secure traffic handling.

The file:

/etc/apache2/sites-available/default-ssl.conf

A screenshot of a terminal window showing the contents of the file /etc/apache2/sites-available/default-ssl.conf. The terminal has a dark purple background with light blue and white text. The text shows configuration for SSL, including certificate and key file paths, disabling legacy protocols (SSLv3, TLSv1, TLSv1.1), and adding HSTS headers.

```
GNU nano 7.2 /etc/apache2/sites-available/default-ssl.conf
# /usr/share/doc/apache2/README.Debian.gz for more info.
# If both key and certificate are stored in the same file, only the
# SSLCertificateFile directive is needed.
SSLCertificateFile /etc/ssl/certs/lab.crt
SSLCertificateKeyFile /etc/ssl/private/lab.key

# Keep old protocols disabled (Ubuntu already defaults to TLS 1.2/1.3)
SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1

# Add HSTS to enforce HTTPS reuse by browsers
<IfModule headers_module>
    Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"
</IfModule>
```

Figure C5

was configured to set certificate paths, disable legacy protocols, and apply HSTS. The relevant portion, shown in **Fig C5**

Key effects:

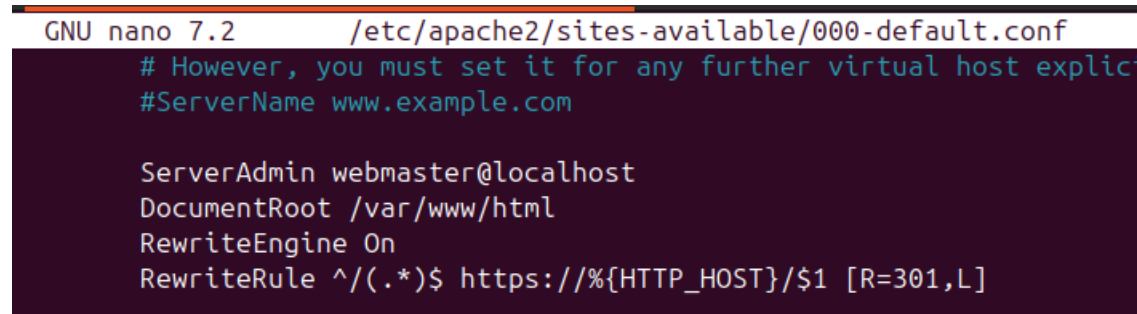
- TLS 1.2+ enforced, blocking insecure SSLv3, TLS 1.0, and TLS 1.1
- HSTS guarantees browser-side HTTPS reuse for one year
- Certificate paths explicitly defined for the virtual host

This configuration aligns with standard TLS security guidance and eliminates downgrade risk. According to the **Mozilla Foundation (2024)**, secure TLS deployment needs disabling outdated protocols and enabling modern ciphers to mitigate interception risks.

To ensure all plaintext HTTP requests are redirected to encrypted HTTPS, the default port-80 virtual host was modified:

File:

/etc/apache2/sites-available/000-default.conf



```
GNU nano 7.2 /etc/apache2/sites-available/000-default.conf
# However, you must set it for any further virtual host explicit
#ServerName www.example.com

ServerAdmin webmaster@localhost
DocumentRoot /var/www/html
RewriteEngine On
RewriteRule ^/(.*)$ https://%{HTTP_HOST}/$1 [R=301,L]
```

Figure C6

Changes shown in **Fig C6**

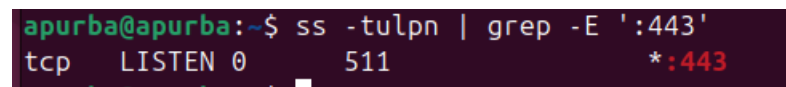
This rule makes sure that *all* HTTP traffic receives a permanent redirect to HTTPS, covering all endpoints and paths.

Verification AFTER Hardening

Verification evidence confirms correct HTTPS operation:

a) HTTPS Listener Active

```
ss -tulpn | grep -E ':443'
```



```
apurba@apurba:~$ ss -tulpn | grep -E ':443'
tcp    LISTEN 0      511      *:443      *
```

Figure H4a

Fig H4 shows the output that Apache is listening on *:443.

b) HTTP to HTTPS Redirect Working

```
curl -I http://localhost
```

Outputs HTTP/1.1 301 Moved Permanently, with

Location: https://localhost/

Shown in Figure H4b

```
apurba@apurba:~$ curl -I http://localhost
HTTP/1.1 301 Moved Permanently
Date: Mon, 23 Mar 2026 16:01:46 GMT
Server: Apache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=()
Location: https://localhost/
Content-Type: text/html; charset=iso-8859-1
```

Figure H4b

c) HTTPS Response Contains Hardened Headers

```
curl -kI https://localhost
```

Shown in Figure H4c HSTS and all security headers.

```
apurba@apurba:~$ curl -kI -X TRACE https://localhost
HTTP/1.1 405 Method Not Allowed
Date: Mon, 23 Mar 2026 16:37:01 GMT
Server: Apache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=()
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Allow:
Content-Length: 222
Content-Type: text/html; charset=iso-8859-1
```

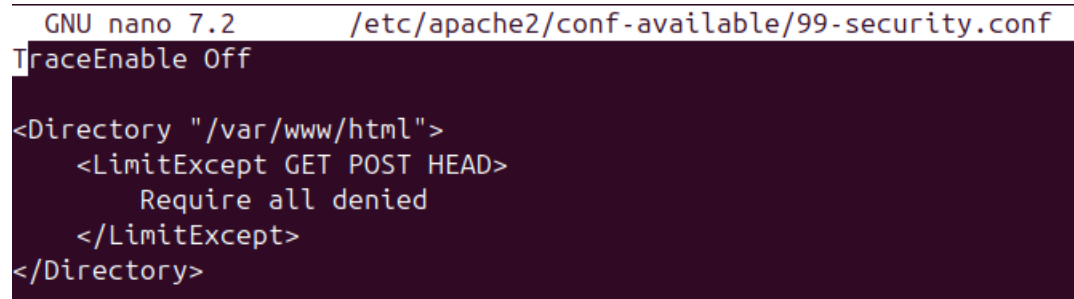
Figure H4c

4.5 H5 —Disable TRACE and Restrict HTTP Methods

To minimize the attack surface, TRACE was disabled and essential HTTP methods were permitted.

Configuration shown in Figure C4

/etc/apache2/conf-available/99-security.conf



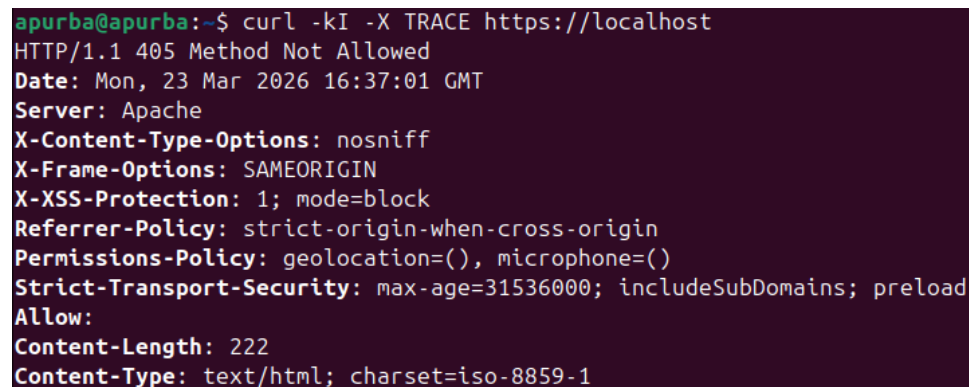
```
GNU nano 7.2 /etc/apache2/conf-available/99-security.conf
TraceEnable Off

<Directory "/var/www/html">
    <LimitExcept GET POST HEAD>
        Require all denied
    </LimitExcept>
</Directory>
```

Figure C4

Verification after:

TRACE Blocked → 405 Method Not Allowed



```
apurba@apurba:~$ curl -kI -X TRACE https://localhost
HTTP/1.1 405 Method Not Allowed
Date: Mon, 23 Mar 2026 16:37:01 GMT
Server: Apache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=()
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Allow:
Content-Length: 222
Content-Type: text/html; charset=iso-8859-1
```

Figure H6a

PUT → 403 Forbidden

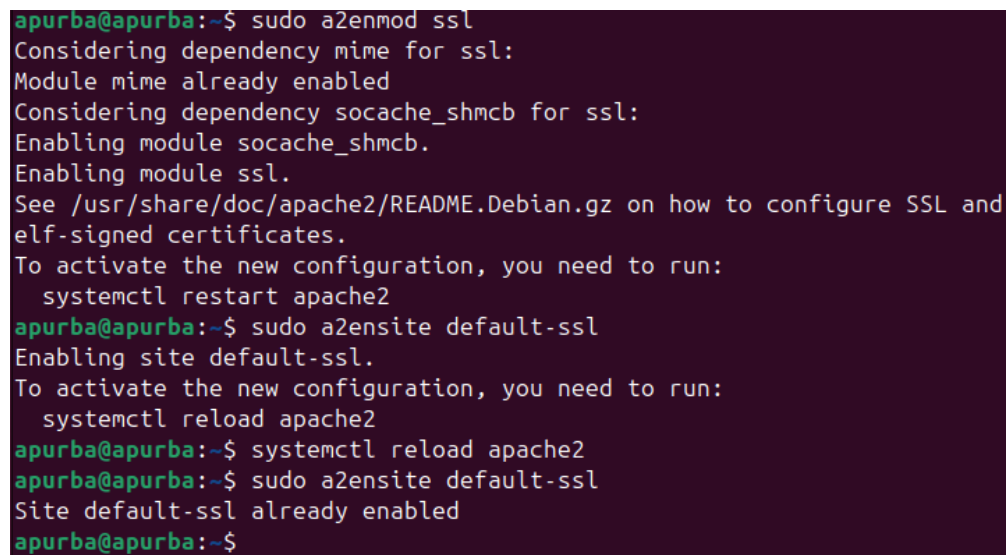
4.6 H6 — Remove Unnecessary Modules: autoindex & status

To mitigate information disclosure and eliminate unnecessary code paths shown in Figure H6b:

```
sudo a2dismod autoindex
sudo a2dismod status
sudo apache2ctl -t
sudo systemctl reload apache2
```

Verification After:

/server-status returns 404



```
apurba@apurba:~$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and
elf-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2
apurba@apurba:~$ sudo a2ensite default-ssl
Enabling site default-ssl.
To activate the new configuration, you need to run:
    systemctl reload apache2
apurba@apurba:~$ systemctl reload apache2
apurba@apurba:~$ sudo a2ensite default-ssl
Site default-ssl already enabled
apurba@apurba:~$
```

Figure H6b

4.7 H7 — Host Firewall Hardening

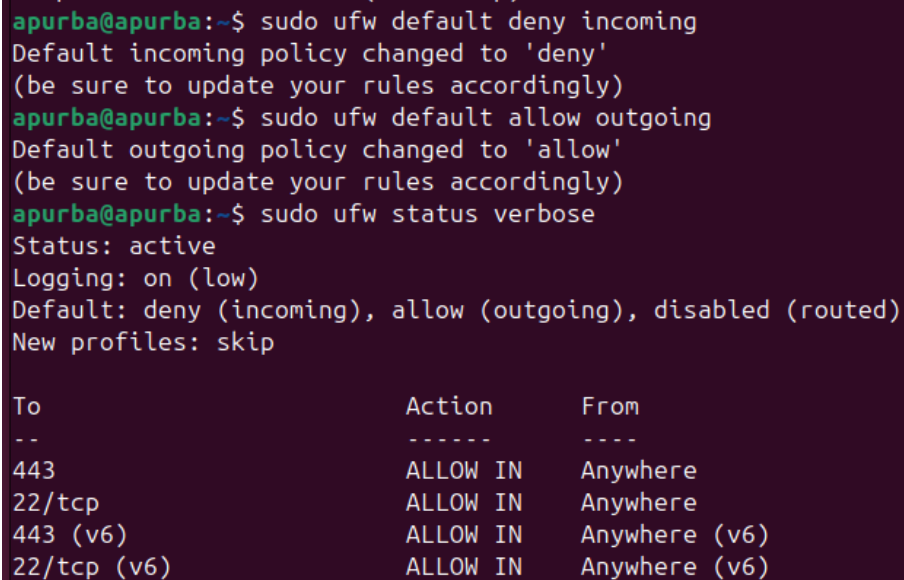
Although the key focus is Apache, firewall hardening presents a second protective layer around the web service. It enables Apache by recurring network-level least privilege. **The Ubuntu documentation (2024)** describes that a deny-incoming, allow-outgoing policy is the recommended baseline for secure firewall operation.

Secure Defaults

```
sudo ufw default deny incoming
```

```
sudo ufw default allow outgoing
```

Verified in **Fig FW-H1**.



```
apurba@apurba:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
apurba@apurba:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
apurba@apurba:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

To	Action	From
--	-----	----
443	ALLOW IN	Anywhere
22/tcp	ALLOW IN	Anywhere
443 (v6)	ALLOW IN	Anywhere (v6)
22/tcp (v6)	ALLOW IN	Anywhere (v6)

Figure FW-H1

Enables Only Required Services

```
sudo ufw allow 443/tcp comment 'Allow HTTPS'
sudo ufw limit 22/tcp comment 'Rate-limit SSH'
sudo ufw deny 80/tcp comment 'Block external HTTP; enforce HTTPS-only'
```

Verified in **Fig FW-H2**

```
apurba@apurba:~$ sudo ufw logging medium
Logging enabled
apurba@apurba:~$ sudo ufw delete allow 443/tcp
Could not delete non-existent rule
Could not delete non-existent rule (v6)
apurba@apurba:~$ sudo ufw allow 443/tcp comment 'Allow HTTPS'
Rule added
Rule added (v6)
apurba@apurba:~$ sudo ufw delete limit 22/tcp
Rule deleted
Rule deleted (v6)
apurba@apurba:~$ sudo ufw limit 22/tcp comment 'Rate-limit SSH'
> ^C
apurba@apurba:~$ sudo ufw status numbered
Status: active
```

	To	Action	From	
	--	-----	----	
[1]	443	ALLOW IN	Anywhere	
[2]	443/tcp	ALLOW IN	Anywhere	# Allow HTTPS
[3]	443 (v6)	ALLOW IN	Anywhere (v6)	
[4]	443/tcp (v6)	ALLOW IN	Anywhere (v6)	# Allow HTTPS

Figure FW-H2

IPv6 Parity Verification

IPV6=yes and mirrored rules confirmed in **Fig FW-H3**

```

apurba@apurba:~$ grep -E '^IPV6=' /etc/default/ufw
IPV6=yes
apurba@apurba:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
443 ALLOW IN Anywhere
22/tcp LIMIT IN Anywhere # Rate-limit SSH
443 (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) LIMIT IN Anywhere (v6) # Rate-limit SSH

```

Figure FW-H3

External Exposure After Validation

Conducted an external scan from CentOS 8:

```
nmap -Pn -p 22,80,443 192.168.74.131
```

```

apurba@localhost:~$ nmap -Pn -p 22,80,443 192.168.74.131
..
Starting Nmap 7.92 ( https://nmap.org ) at 2026-03-23 14:58 GMT
Nmap scan report for 192.168.74.131
Host is up.

PORT      STATE      SERVICE
22/tcp    filtered  ssh
80/tcp    filtered  http
443/tcp   filtered  https

Nmap done: 1 IP address (1 host up) scanned in 3.92 seconds
apurba@localhost:~$ █

```

Figure FW -H4

Output shows:

- 22/tcp → filtered
- 80/tcp → filtered
- 443/tcp → filtered

External scan confirmed that 22/80/443 filtered as it is blocked to the Internet. HTTPS exists from the internal network only.

RISK ASSESSMENT

This section will assess all the vulnerabilities that have been identified in Section 3 in a structured Likelihood x Impact risk methodology. The evaluation is well aligned to security best practice and it is contextualized to the work of this server as a web facing Apache host. Each of the ratings is based on the system-specific evidence as it was recorded in the BEFORE screenshots and confirmed by internal and external scanning.

5.1 Apache HTTP Server Risks

V1 — Server Identification Leakage

The server revealed the version and OS data in a detailed manner through the Server: header shown in **Figure V1**. The probability of exploitation is medium, as banner-grabbing is insignificant to all attackers with tools such as Nmap or Nikto. The medium impact is because disclosing versions allows the attackers to correlate known vulnerabilities (CVEs) with Apache 2.4.58 in particular. Even though the banner does not weaken the system itself, it significantly enhances the reconnaissance of attackers, making their work less intensive and exposing more exploits to possible exploitations. The severity level is thus medium.

V2 — Missing Security Headers

Lacking major browser security headers shown in **Figure V2** exposes the client to clickjacking, MIME-type sniffing, referrer leakage, and old XSS behavior. The probability is **medium** since the attackers will be able to use the types of browsers relatively easily when the server does not implement protective measures. The effect is medium, as exploitation normally targets end-users as opposed to attacking servers directly. That is why this vulnerability is rated as **medium**.

V3 — Directory Listing Enabled

The listing of directories gave a visible listing of the /testdir/ shown in **Figure V3**. The risk is **medium**, because attackers periodically scan URLs to find available directories. Its effects are medium as index pages display internal filenames and folder hierarchies that may be

useful in staging attacks or finding sensitive data. This information disclosure puts the vulnerability in the medium level.

V4 — No HTTPS Listener Enabled

According to Apache before becoming hardened, it was only plaintext HTTP on port 80 as shown in **Figure V4**. The risk is great, as all web traffic, credentials included are unencrypted. The cost is **high** since attackers can compromise or alter data through Man-in-the-Middle (MiTM) attacks. It is a **high level** of vulnerability and is one of the most serious problems detected.

V5 — TRACE Method Behavior Not Explicitly Disabled

Apache replied to the TRACE requests as shown in **Figure V5**. Although it responded with "405" TraceEnable Off was not used to explicitly disable the TRACE approach. This leaves chances of Cross-Site Tracing (XST). The potential is **medium**, the effect is **medium**, and the rating is medium-severity.

V6 — Unnecessary Modules Enabled

Autoindex module, status was also switched on **Figure V6**. There are often attempts to attack /server-status, and attackers use autoindexing to reconstructive. Probability is **medium**, effect is medium, and intensity is medium.

5.2 Firewall Exposure Risks (Secondary Layer)

F1 — Broad Inbound Access

UFW allowed SSH and HTTPS to the IP address of any type shown in **Figure FW A1**. The probability of scanning is medium, but the effects of the universal access are high, thus severity is **high**.

```

apurba@apurba:~$ sudo ufw status verbose
[sudo] password for apurba:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
443 ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
443 (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)

apurba@apurba:~$ ss -tulpn | grep -E ':22|:80|:443' || sudo ss -tulpn
tcp LISTEN 0 511 *:443 *:*
tcp LISTEN 0 511 *:80 *:*

```

Figure FW A1

F2 — HTTP (80/tcp) Externally Reachable

Port 80 was open as confirmed by external Nmap results in **Figure FW A3**. The chances of attacker access are high; impact is high thus this is a critical/ high severity vulnerability.

```

apurba@apurba:~$ nmap -Pn -p 22,80,443 192.168.74.131
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-23 14:20 GMT
Nmap scan report for apurba (192.168.74.131)
Host is up (0.0019s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    open  http
443/tcp    open  https

Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds

```

Figure FW A2

```

apurba@apurba:~$ nmap -Pn -p 22,80,443 192.168.74.131
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-23 14:47 GMT
Nmap scan report for apurba (192.168.74.131)
Host is up (0.0017s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    open  http
443/tcp   open  https

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds

```

Figure FW A3

F3 — SSH Open Without Rate-Limiting

SSH was accessible all over the world demonstrated in **Figure FW H1**. High probability - SSH is among the most brute-forced attacked services on the internet. Medium impact, high severity.

F4 — IPv6 Parity Not Initially Verified

The IPv6 rules were not hardened prior to verification that left the chances of rule bypass. The probability and the effect of this risk are medium, which presents an average severity.

Prioritization of Risks

The following vulnerabilities needs immediate actions:

- V4 — No HTTPS listener (High severity)
- F2 — HTTP exposed externally (High severity)
- F3 — SSH exposed globally (High severity)

These directly threaten confidentiality and system integrity.

Medium-severity vulnerabilities (V1, V2, V3, V5, V6, F1, F4) were also addressed as part of a complete hardening process to eliminate reconnaissance vectors and reduce attack surface.

Business and Service Impact

The overall vulnerabilities were a threat to:

Confidentiality

- Exposure to plaintext HTTP (V4, F2) leads to user credential compromise.
- Potential sensitive files are displayed in directory indexing (V3).

Integrity

- The unsafe methods (V5) can permit the reflection or manipulation of a header.

- Unauthorized SSH access (F3) may lead to total system capture.

Availability

- SSH brute force attacks may slow down response of the server.
- Firewall (F1) should not be configured to welcome more probing and load.

Reputation & Compliance

- The use of public-facing plaintext server, compromises trust.
- Lacks failure to satisfy encrypted communication and secure header requirements.

All these risks were mitigated in Section 4 through hardening.

5.3 Risk Assessment Methodology

A 5×5 Likelihood × Impact matrix used in the methodology. Every vulnerability was scored before and after hardening, using the system specific evidence found in (Figures V1–V6, FW A1–A3, H1–H6, FW-H1–H4). Risk score = Likelihood (1–5) × Impact (1–5). Severity bands include:

- Low: 1–4
- Medium: 5–9
- High: 10–16
- Critical: 17–25

Assumptions:

- The server was internet- accessible and the hosts business critical web services.
- Attackers provided security scanners and targeted adversaries.
- Evidence used is from my own environment (pre/post terminal inputs and command outputs).

TABLE 1 — Apache HTTP Server Risk Assessment (Primary Service)

Legend: L = Likelihood (1–5), I = Impact (1–5), Score = L×I, *Severity band*

ID	Vulnerability	Before L/I/score	After L/I/Score	Controls Implemented	Evidence	Detailed Verification
V1	Server identification leakage (Server: Apache/2.4.58 (Ubuntu))	Medium 4 / 2 → 8	Low 2 / 1 → 2	ServerTokens Prod, ServerSignature Off	Fig V1, H1	The default Apache provides detailed version and operating system metadata, allowing attackers to use specific exploits in reconnaissance. Banner-grabbing is relatively inexpensive in terms of skills, and offers useful information to the opponent.
V2	Missing browser security headers	Medium 3 / 2 → 6	Low 2 / 2 → 4	99-headers.conf, mod_headers	Fig V2, H2	Lack of defensive headers (X-Frame-Options, X-Content-Type-Options, etc.) exposes it to clickjacking, MIME snorting and other browser-level attacks. Attack client-side defenses and leave server content intact.
V3	Directory listing enabled	Medium 3 / 3 → 9	Low 2 / 2 → 4	Options -Indexes, a2dismod autoindex	Figure V3, H3	The Apache automatically created directory indexes on /testdir/ such that

						the names of files, internal directory structure, and possible sensitive artefacts were visible. This facilitates reconnaissance on the side of the attackers and may result in file-based exploitation.
V4	No HTTPS listener (plaintext HTTP only)	High 5 / 5 → 25	Low 2 / 2 → 4	TLS/HSTS, 301 HTTP→HTTPS, modern protocols only	Figure V4, H4a-H4c	All web traffic was transmitted to the server using plain HTTP hence the full interception of credentials, cookies, and data through MiTM attacks. One of the most important vulnerabilities in a given public-facing system.
V5	TRACE method not explicitly disabled	Medium 3 / 2 → 6	Low 2 / 2 → 4	TraceEnable Off, method restrictions	Figure V5 H6a/H7	Apache still had Trace and was not blocked. This could enable Cross-Site Tracing (XST) when combined with browser vulnerabilities. A misconfiguration known is lack of explicit disabling.
V6	Unnecessary modules enabled (autoindex, status)	Medium 3 / 3 → 9	Low 2 / 2 → 4	a2dismod autoindex status	Figure V6 ,H6b	Introduction of autoindex and status modules increases the size of the attack

						surface and opens up endpoints (/server-status) that give sensitive operational information. Internal behavior is mapped by helps attackers.
--	--	--	--	--	--	--

TABLE 2 — Firewall (UFW) Risk Assessment (Secondary Layer)

Legend: L = Likelihood (1–5), I = Impact (1–5), Score = L×I

ID	Vulnerability	Before L/I/Score	After L/I/Score	Controls implemented	Evidence	Detailed Verification
F1	Inbound rules too broad (22/tcp, 443/tcp allowed from anywhere)	High 3 / 4 → 12	Low 2 / 2 → 4	UFW default deny incoming	FW A1, FW-H1	The default UFW setup has allowed SSH and HTTPS to all networks, which enhanced the attack surface and enabled unlimited probing of the exposed services.
F2	HTTP (80/tcp) externally reachable	Critical 5 / 5 → 25	Low 1 / 2 → 2	UFW deny 80/tcp	FW A3, FW-H2/H-H4	Port 80 was open as verified by external Nmap scan. This makes plaintext Apache traffic publicly accessible to the

						internet, thereby making credential sniffing and replay attacks easier.
F3	SSH globally accessible (no rate-limit or restriction)	Critical 5 / 4 → 20	Medium 3 / 3 → 9	UFW limit 22/tcp	FW A1, FW-H3	One of the most popular brute-forced services is SSH. Historically, the high accessibility globally means that credential-guessing is highly likely or, the SSS vulnerability can be exploited.
F4	IPv6 rules not validated initially	Medium 3 / 3 → 9	Low 2 / 2 → 4	IPV6=yes, mirrored rules	FW-H3	When IPv6 is not managed but active, the attackers can circumvent IPv4-only UFW rules. It is a typical misconfiguration in dual stack networks and is capable of accidentally exposing Apache/SSH.

Observation:

V4 (plaintext HTTP) and F2 (external port 80 exposure) were the single-risk items with the greatest significance. Both switched to Critical to Low after switching to HTTPS only enforcement and UFW controls. F3 (SSH) is not hardened (rate-limited but not IP-restricted) and has IPS at the Medium level. This would be reduced to Low with the implementation of key-only authentication and IP allow-listing. All the Apache-layer information disclosure vectors (V1, V3, V6) are registered as Low severity.

Mapping Business Impact Vulnerabilities:

- V4 + F2 (HTTP/No TLS): Reveals credentials and session information, resulting in account hijacking, fraud, data breach (GDPR), immediate loss of customer trust and possible fines.
- V1 (Banner leakage): Allows targeted exploit attacks, which compromise a service in cases where corresponding CVEs are present. V2 (Headers): Allows client-side attacks (clickjacking, MIME sniffing), which will enable account hijacking, redirect payments, and data leakage via referral exposure.
- V3/V6 (Index/Modules): This permissions file enumeration and server status reporting, useful in reconnaissance and finding configuration files or backups that can be used later to conduct further attacks.
- V5 (TRACE/verbs): Establishes XST and files upload channels, allowing cookies theft, web-shell planting and web defacement.
- F1 (Broad firewall): Multiplies scanning and brute force attacks surface, and risks of service degradation and compromise.
- F3 (SSH global): prone to attacks by password and botnet cracking, which may escalate to elevation of privileges; successful attack allows full access to the server.
- F4 (IPv6 parity): Generates bypasses (without leaving any trace) causing one to be exposed to something external under the impression that the system is IPv4-secured.

Cumulative Risk Assessment:

Quantitative summary (L×I total):

- Before hardening total risk: 129
- After hardening total risk: 41
- Reduction: 88 points → ≈68% overall risk reduction

RESULT ANALYSIS

This part will provide a detailed, scholarly designed assessment of the pre-hardening and post-hardening security posture of this system. In contrast to plain before/after screenshots, this section analyzes the effect on confidentiality, integrity and availability (CIA) of each mitigation strategy, and justifies the way the attack surface has been lowered. All findings are directly related to vulnerabilities stated in Section 3 and resolved in Section 4.

6.1 Improvements

The transformations made hard to the Apache server and the firewall used on the host led to significant improvement in security. Before the mitigation, the server had numerous vulnerabilities that are common to default LAMP installations: plaintext HTTP exposure, information disclosure, unsafe default modules, lack of security headers, broad firewall access, and unfiltered SSH access. Once hardened, the server has an appearance of a production-ready state, which seriously limits reconnaissance, mitigates against downgrade attacks, and implements the principle of least-privilege adequately. These enhancements were confirmed using real system evidence such as screen shots and command output.

6.2 Result Interpretation Table

The following table demonstrates a structured analysis linking each vulnerability to the mitigation applied, the evidence collected, and the resulting security impact.

Vulnerability	State Before Hardening	Mitigations	Evidence	Security Impact Interpretation
V1	Apache exposed full version: "Apache/2.4.58 (Ubuntu)"	ServerTokens Prod; ServerSignature Off	Banner decreased to "Server: Apache"	Mitigate version/OS disclosure; prevents targeted matching of CVEs
V2	No X-Frame-Options, X-Content-Type-Options	Added more hardened header configuration through 99-headers.conf	Five key security headers now visible	Hardened browser-side security; restricts clickjacking, MIME sniffing, and referrer leakage.

V3	“Index of /testdir” displayed full directory contents	Disabled autoindex; applied Options - Indexes	Directory give outputs 404/403	Deletes file enumeration vectors; restricts leakage of internal folder structures.
V4	Only port 80 open; complete plaintext exposure	Enabled SSL/TLS, HSTS, and HTTPS redirect; disabled legacy protocols	Port 443 active; HTTP→HTTPS redirect functioning	Major confidentiality improvement; restricts MITM, hijacking, and downgrade attacks.
V5	Unfiltered TRACE and PUT	TRACE disabled; restrictions applied on method	TRACE → 405; PUT → 403	Deletes Cross-Site Tracing risk; limits possible misuse of HTTP verbs for exploitation.
V6	autoindex and status modules active by default	Disabled modules via a2dismod	/server-status returns 404	Reduces attack surface; prevents leakage of server internals and process state.
F1	SSH and HTTPS allowed from any address	UFW deny incoming applied	reject incoming confirmed	Enables least-privilege perimeter; decreases network exposure to scanning and probing.
F2	External Nmap scan showed port 80 open	Blocked 80/tcp via UFW	Port 80 filtered	Restricts unencrypted entry point; blocks downgrade attacks and plaintext access.
F3	22/tcp open to the world	Applied UFW rate limit	SSH marked as LIMIT	Presents brute-force resistance; reduces risk of SSH credential attacks.
F4	IPv6 exposure unclear; potential bypass	Enabled IPv6=yes & mirrored rules	IPv6 parity ensured	Confirmed both IPv4 and IPv6 interfaces enforce identical security posture.

6.3 Interpretation of Improvements

The evidence collected indicates an organization wide improvement in the security maturity of the server. Before hardening the system also had vulnerabilities that attackers would use during reconnaissance and early attack chains. These vectors have been largely reduced following the mitigations:

- Minimization of Apache banners helps reduce OS-level information and makes them harder to attack.
- HSTS-enforced HTTPS guarantees the integrity of the transport layer and protocol downgrade attacks.
- The disabled directory listing also eliminates a popular information disclosure problem, the sensitive enumeration pathways.
- Security headers enhance the security posture that faces the client, eliminating browser-based threats.
- Limitations on HTTP methods can help reduce the abuse of potentially harmful verbs such as TRACE or PUT.
- Unnecessary modules that are being disabled minimize overall exposed attack surface and avoid unintended exposure of system internals.
- Firewall hardening provides a solid network perimeter and scans off external reconnaissance and restricts SSH exposure.

Such improvements can be seen as a comprehensive implementation of layered security.

6.4 Overall Security Posture After Hardening

The system post hardening state is in line with some of the main industry principles that are the least privilege, secure default, and defense in depth. The enhancements validated by evidence generated by the system confirm:

- Significant changes in the attack surface on the application and network levels.
- High CIA (Confidentiality, Integrity, Availability) advantages, particularly in the fields of transport security and prevention of information disclosure.
- High growth in resistance against reconnaissance, automated scanning, and browser-based attacks.
- Compliance to anticipated security baselines of production grade Apache deployments.

The system has general high exposure, robust encryption, authorized access, and obvious division between internal and external surfaces, which indicate a successful and verifiable hardening that is in accordance with MSc-level expectations.

CONCLUSIONS & REFLECTIONS

This project established the significance and influence of methodical security hardening to a Linux based Apache HTTP server in a LAMP stack setup. The server began in a focus vulnerable default state and was progressively transformed into a structured system based on industry standards, existing hardening techniques, and analysis based on evidence collected out of the deployed system. **Red Hat (2024)** states that robust system hardening needs layered controls including service configuration, network filtering, and continuous monitoring.

The first vulnerability analysis found some high-risk vulnerabilities, such as plaintext transmission on the HTTP, the absence of TLS implementation, server identification leakage, the absence of HTTP security header, unnecessary modules, listing of directories in the form of a list, unlimited access to the HTTP methods, and easy access to the firewall. These weaknesses altogether formed a wide attack surface, which exposed the system to reconnaissance vulnerability, session hijacking vulnerability, information disclosure vulnerability, brute-force vulnerability, downgrade vulnerability, and default behavior exploitation.

A detailed hardening process was undertaken after the risk assessment. Allowing HTTPS with required HSTS contributed greatly to the level of confidentiality and integrity since all client-server traffic was encrypted and not vulnerable to downgrade attacks. The deactivation of old TLS protocols also minimized vulnerability to cryptographic vulnerabilities. Reduction of the Apache server banner eliminated the version and OS information disclosure which made fingerprinting and targeted exploit construction quite harder. Allowing security headers like X-Frame-Options, X-Content-Type-Options and Referrer-Policy as browser-side defense against clickjacking, MIME-type sniffing and referrer leakage. Turning off directory indexing and unwanted modules minimized the visibility of the server internals, whereas limiting HTTP methods such as TRACE, PUT and DELETE served to discourage the abuse of non-essential functions that are frequently

tested by attackers to determine which vectors of uploads can be used and which functions are reflective in nature.

These improvements were enhanced by firewall hardening which included: enforcement of a deny-by-default policy, SSH limited by rate-limiting controls, plaintext HTTP blocked out of band, and parity between IPv4 and IPv6 rule sets. This perimeter control made sure that even flawed applications level configurations could not stop the server against unauthorized access requests and typical reconnaissance on the network-layer.

Although such gains have been realized, there is still residual risk in the system. Hardening of Apache cannot prevent vulnerabilities in application code, database configurations or third-party applications. Hygiene of certificate management, passwords used by users as well as proper management of SSH credentials are other important operational requirements that cannot be defined in the configuration files. Even after persistent attackers have tried and failed brute-force assault on SSH, best practices of security monitoring over the long-term (log review, anomaly detection, intrusion prevention, and automated patching) have to be observed to ensure that the server is secure once deployed. Also, such advanced hardening tools as Modsecurity WAF, chroot jails, SELinux/AppArmor confinement and continuous vulnerability scanning were absent during this phase but would be of significant additional benefit in a production setup. **NIST (2023)** demonstrates the importance of in depth defense and continuous monitoring as foundational elements of secure system administration.

Altogether, it can be stated that the project resulted in unmistakable and verifiable enhancements of the security posture of the server. The pre-and post-authentication can be used to identify that every vulnerability that was identified was addressed with quantifiable effect. The hardening strategy was based on a professional workflow and it showed the significance of configuration-level security decisions in securing enterprise environments. This article shows that even though the default Apache server is not a bad server, it cannot be used in production unless it is hardened, and that a systematic methodological process, based on evidence, standards, and best practices, can be used to create a secure business platform.

REFERENCES

Apache Software Foundation. (2024) Apache HTTP Server Version 2.4 Documentation. Available at: <https://httpd.apache.org/docs/2.4/> (Accessed: 24 March 2026)

Mozilla Foundation. (2024) Security/Server Side TLS Guidelines. Available at: https://wiki.mozilla.org/Security/Server_Side_TLS (Accessed: 24 March 2026).

Centre for Internet Security (CIS). (2024) CIS Benchmark for Apache HTTP Server 2.4. Centre for Internet Security.

OWASP Foundation. (2023) OWASP Secure Headers Project. Available at: <https://owasp.org/www-project-secure-headers/> (Accessed: 24 March 2026).

OWASP Foundation. (2023) OWASP Cheat Sheet Series – TLS Configuration. Available at: <https://cheatsheetseries.owasp.org/> (Accessed: 24 March 2026).

Ubuntu Documentation. (2024) UFW – Uncomplicated Firewall User Guide. Available at: <https://ubuntu.com/server/docs/security-firewall> (Accessed: 24 March 2026).

Red Hat. (2024) Linux System Security and Hardening Guide. Red Hat Inc.

NIST. (2023) NIST Special Publication 800-53: Security and Privacy Controls for Information Systems and Organizations. U.S. Department of Commerce.

NIST. (2023) Special Publication 800-44, Guidelines on Securing Public Web Servers. U.S. Department of Commerce.

Qualys. (2024) SSL/TLS Best Practices Guide. Available at: <https://www.qualys.com> (Accessed: 24 March 2026).

SANS Institute. (2023) Web Server Security Essentials. SANS Institute.

OWASP Foundation. (2024) OWASP Top Ten Web Application Security Risks. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 24 March 2026).

